



预训练语言模型

目录

CONTENTS

- 1 概述**
- 2 自回归预训练模型代表：GPT**
- 3 自编码预训练模型代表：BERT**
- 4 预训练语言模型的应用**
- 5 深入理解BERT**

目录

CONTENTS

1

概述

2

自回归预训练模型代表：GPT

3

自编码预训练模型代表：BERT

4

预训练语言模型的应用

5

深入理解BERT

动态词向量模型的问题

□数据

- 训练数据相对比较局限，例如CoVe要求使用双语平行句对

□模型

- 表示模型的参数量相对较小（相比预训练语言模型），模型深度不够

□用法

- 通常使用这类模型时，表示模型本身是不参与训练的（权重无更新）
- 表示模型本身不参与训练，一定程度上限制了表示模型在下游任务上的泛化能力

Pretrain

- 计算机视觉 (Computer Vision , CV) 领域
- 大规模数据集:ImageNet
- 预训练：通常会使用ImageNet进行一次预训练
 - 让模型从海量图像中充分学习如何从图像中提取特征
 - 根据具体的目标任务，使用相应的领域数据精调，使模型适配领域

Pretrain

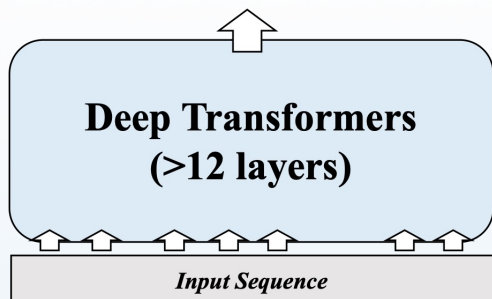
- 领域：NLP
- 大规模数据集:大规模的文本数据
- 预训练目的：预训练语言模型
 - 静态词向量模型：Word2vec、GloVe
 - 动态词向量模型: 基于上下文建模的CoVe、ELMo等
 - 2018年，以GPT和BERT为代表的基于深层Transformer的表示模型出现后，预训练语言模型这个词才真正被大家熟知
 - 目前在NLP领域中提到的预训练语言模型大多指此类模型
 - 预训练语言模型的出现使得自然语言处理进入新的时代，也被认为是近些年来NLP领域中的里程碑事件

预训练模型三要素

大数据
(无标注文本)



大模型
(深度神经网络)



大算力
(并行计算集群)



GPT

GPT-2

GPT-3

以及更多
钱



预训练模型三要素

□大模型(1/3)

□模型需能容纳大数据

□数据规模和模型规模在一定程度上是正相关的

- 当在小数据上训练模型时，通常模型的规模不会太大，以避免出现过拟合现象

- 当在大数据上训练模型时，如果不增大模型规模，可能会造成新的知识无法存放的情况，从而无法完全涵盖大数据中丰富的语义信息

□需要一个容量足够大的模型来学习和存放大数据中的各种特征

□“容量大”通常指的是模型的“参数量大”

预训练模型三要素

□大模型(2/3)

□如何设计这样一个参数量较大的模型呢？主要考虑两个方面

- ① 具有较高的并行程度，弥补大模型带来的训练速度下降的问题
- ② 能够捕获并构建上下文信息，以充分挖掘大数据文本中丰富的语义信息

□构建PLM的最佳选择：基于Transformer的神经网络模型

预训练模型三要素

□大模型(3/3)

□构建PLM的最佳选择：基于Transformer的神经网络模型

① Transformer模型具有较高的并行程度

□Transformer核心部分的多头自注意力机制不依赖于顺序建模，可快速地并行处理。

□传统的神经网络语言模型通常基于RNN，RNN需按序列顺序处理，并行化程度较低

② Transformer中的多头自注意力机制能够有效地捕获不同词之间的关联程度，并且能够通过多头机制从不同维度刻画这种关联程度，使得模型能够得到更加精准的计算结果

□主流的预训练语言模型无一例外都使用了Transformer作为模型的主体结构

预训练模型三要素

□ 大数据

□ 大规模文本数据：训练一个好的预训练语言模型的开始

□ 预训练数据需要讲究

- “保质”：质量要尽可能高，避免混入过多的低质量语料

- “保量”：规模要尽可能大，从而获取更丰富的上下文信息

□ 在实际情况中，预训练数据往往来源不同

□ 精细化地预处理所有不同来源的数据是非常困难的

- 在预训练数据的准备过程中，预处理语料的共性问题，通常不会进行非常精细化地处理

- 通过增大语料规模进一步稀释低质量语料的比重，降低质量较差的语料对预训练过程带来的负面影响

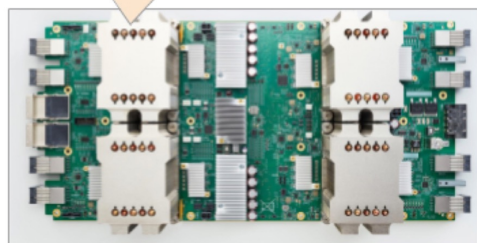
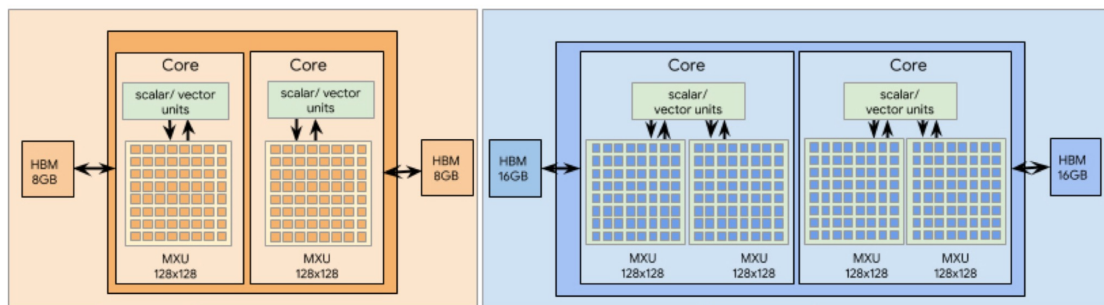
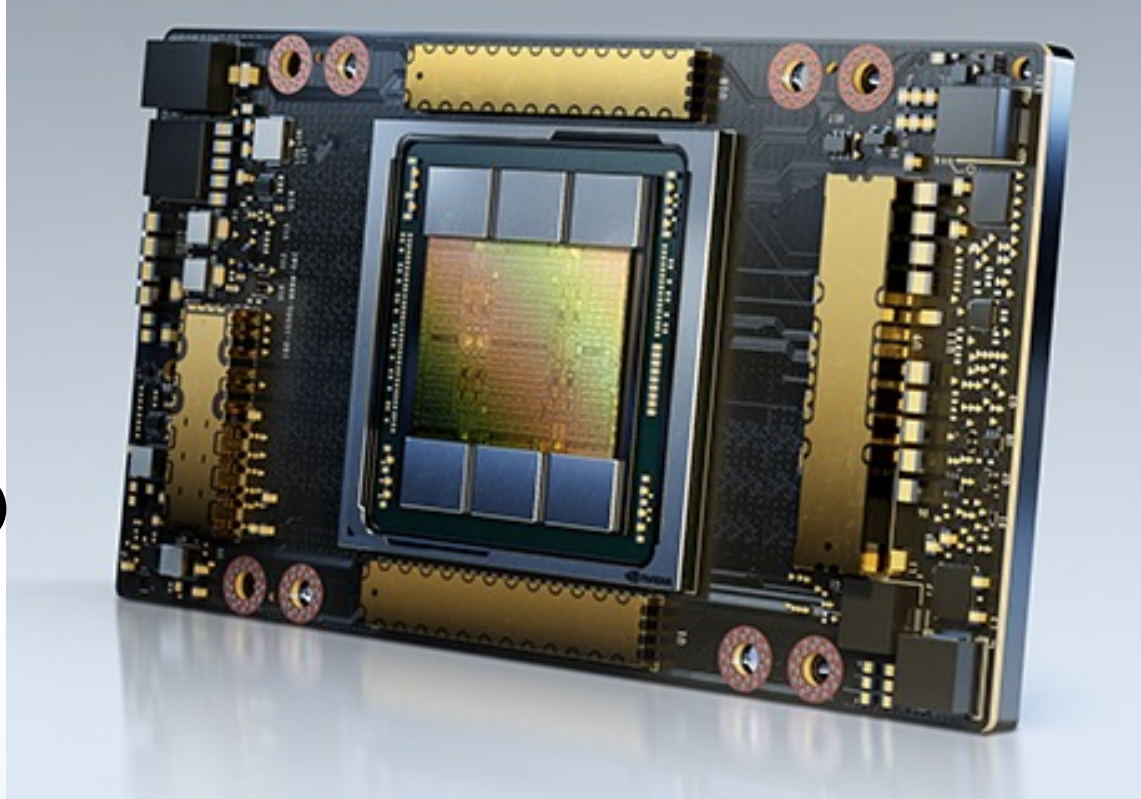
□ 预训练语料的质量越高，训练出来的预训练语言模型的质量也相对更好，这需要在数据处理投入和数据质量之间做出权衡

预训练模型三要素

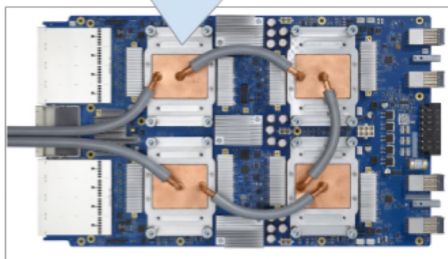
大算力

常见计算设备

图形运算单元 (GPU)



TPU v2 - 4 chips, 2 cores per chip



TPU v3 - 4 chips, 2 cores per chip

张量运算单元 (TPU)

预训练模型三要素

□大算力——GPU

□GPU功能：

- 早期用来处理计算机图形，连接计算机主机和显示终端的纽带
- 随着GPU核心的不断升级，在其计算能力和计算速度得到大幅提升后，不仅可以作为常规的图形处理设备，同时也可以成为深度学习领域的计算设备

□不使用CPU运行深度学习任务原因？

- CPU擅长:处理串行运算以及逻辑控制和跳转
- GPU擅长:大规模并行运算
- 深度学习中经常涉及大量的矩阵或张量之间的计算，并且这些计算是可以并行完成的，特别适合用GPU处理

□GPU品牌：英伟达(NVIDIA)

预训练模型三要素

□大算力——GPU

□GPU品牌：英伟达(NVIDIA)

- 英伟达生产的GPU依靠与之匹配的统一计算设备架构(CUDA)能够更好地处理复杂的计算问题，同时深度优化多种深度学习基本运算指令。
- PyTorch、TensorFlow等主流的深度学习框架均提供了基于CUDA的GPU运算支持，并提供了更高层、更抽象的调用方式，使得用户可以更方便地编写深度学习程序
- 目前广受欢迎的深度学习设备是英伟达Volta系列硬件，其中最为人熟知的型号是V100
 - 深度学习框架下的浮点运算性能达到了125 TFLOPS (以NVLink版为例)
 - 人工智能推理吞吐量比CPU高出20倍以上，并且在高性能计算方面相比CPU高出100倍以上

预训练模型三要素

□大算力——GPU

□GPU品牌：英伟达(NVIDIA)

- 英伟达生产的GPU依靠与之匹配的统一计算设备架构(CUDA)能够更好地处理复杂的计算问题，同时深度优化多种深度学习基本运算指令。
- PyTorch、TensorFlow等主流的深度学习框架均提供了基于CUDA的GPU运算支持，并提供了更高层、更抽象的调用方式，使得用户可以更方便地编写深度学习程序
- 目前广受欢迎的深度学习设备是英伟达Volta系列硬件，其中最为人熟知的型号是V100
 - 深度学习框架下的浮点运算性能达到了125 TFLOPS (以NVLink版为例)
 - 人工智能推理吞吐量比CPU高出20倍以上，并且在高性能计算方面相比CPU高出100倍以上

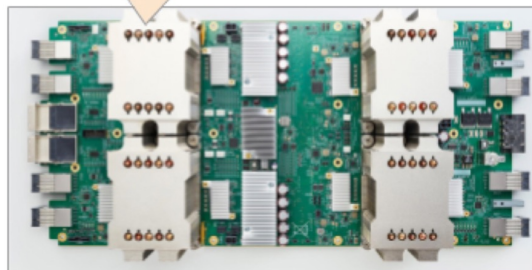
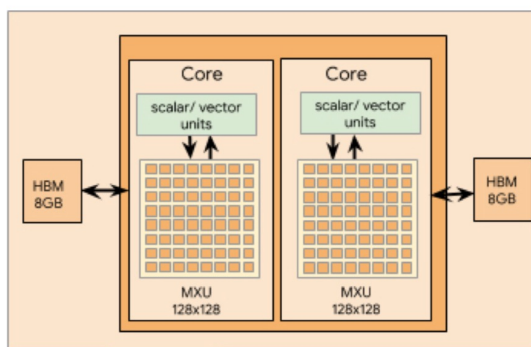
预训练模型三要素

❑大算力——TPU

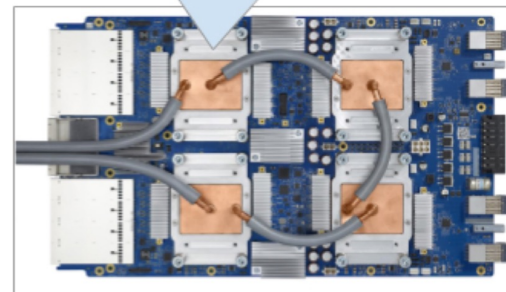
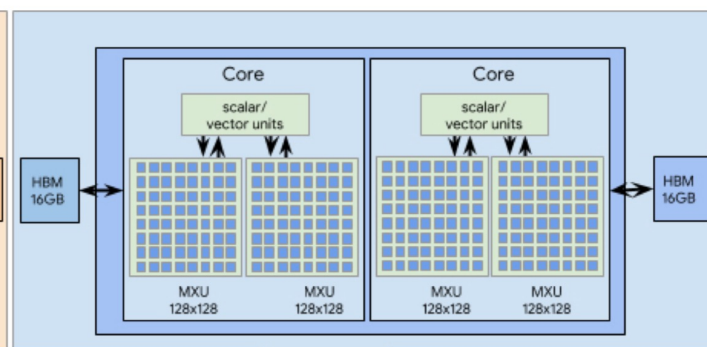
- ❑谷歌公司近年定制开发的专用集成电路，专门用于加快机器学习任务的训练
- ❑能够使用TensorFlow在TPU加速器硬件上快速地完成机器学习任务的训练
- ❑TPU提高了机器学习应用中大量使用的线性代数计算的性能
- ❑当训练大型复杂的神经网络模型时，TPU可以大幅度缩短达到既定准确率所需的时间，提高模型的收敛速度
 - ❑以前在其他硬件平台上需要花费数周时间进行训练的深度学习模型，在TPU上只需数小时即可完成训练
 - ❑借助谷歌公司开发的TensorFlow深度学习框架以及对TPU硬件的针对性优化，可以借助TensorFlow提供的API，方便地将模型迁移到TPU硬件上运行
- ❑TPU主要支持TensorFlow深度学习框架，并逐步完善对PyTorch深度学习框架的支持

TPU v2和TPU v3

型号	芯片数	每芯片核心数	每核心内存	总内存	浮点运算能力
TPU v2	4	2	8 GB	64 GB	180 TFLOPS
TPU v3	4	2	16 GB	128 GB	420 TFLOPS



TPU v2 - 4 chips, 2 cores per chip



TPU v3 - 4 chips, 2 cores per chip

预训练模型三要素

❑大算力——TPU

- ❑与分布式GPU类似，谷歌数据中心中的TPU Pod是通过专用高速网络相互连接的多TPU设备，也是目前训练超大规模预训练语言模型的首选设备之一
- ❑TPU只能通过谷歌云服务器访问使用，无法像GPU一样自行采购使用。
- ❑一张TPU v2的每小时使用费用是4.5美元，而TPU v3是8美元，价格较为昂贵。
- ❑不过，对于想体验TPU的用户来说，谷歌公司推出的Colab在线编程平台是一个很好的选择。
- ❑Colab是一个基于Jupyter Notebook的可交互式在线编程平台，目前用户可以免费使用Colab的基础版本。
- ❑用户可以选择一张英伟达GPU或者谷歌TPU做深度学习相关的实验。
- ❑感兴趣的读者可以访问Colab相关网站了解更多详情。

目录

CONTENTS

1

概述

2

自回归预训练模型代表：GPT

3

自编码预训练模型代表：BERT

4

预训练语言模型的应用

5

深入理解BERT

□ GPT: Generative Pre-Training

- OpenAI提出了“生成式预训练+判别式精调”框架
- 正式开启了自然语言处理领域“**预训练+精调**”的新时代

生成式预训练

- 在大规模文本数据上训练一个高容量的语言模型，从而学习更加丰富的上下文信息

判别式任务精调

- 将预训练好的模型适配到下游任务中，并使用有标注数据学习判别式任务

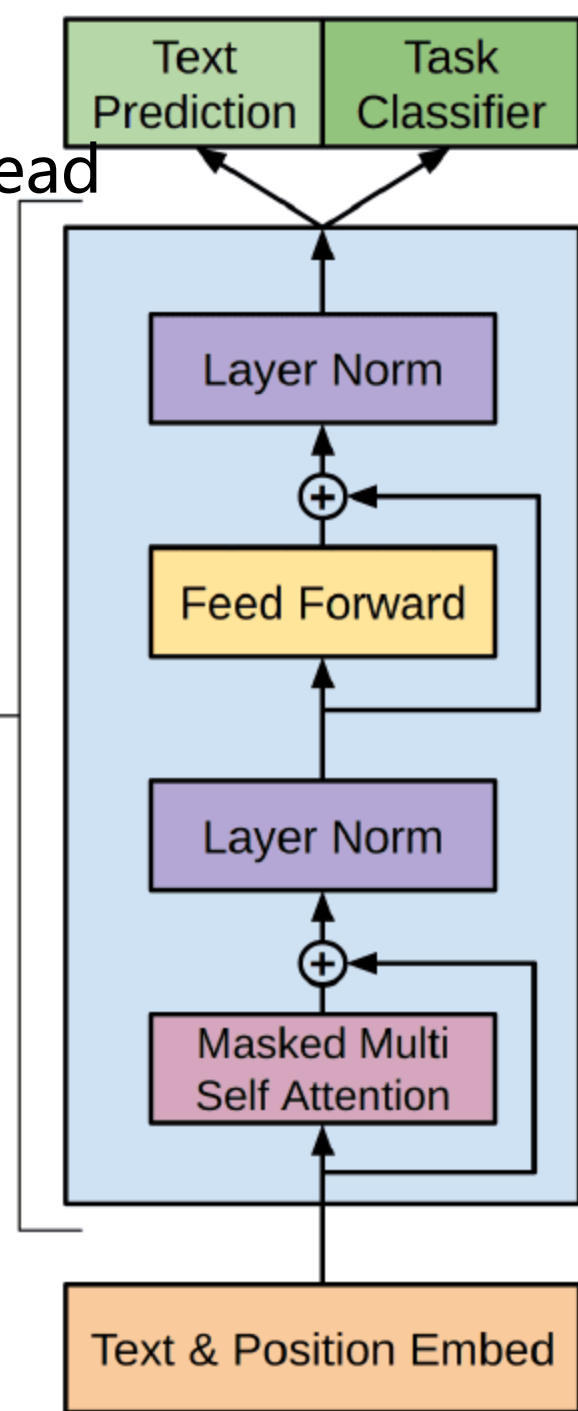
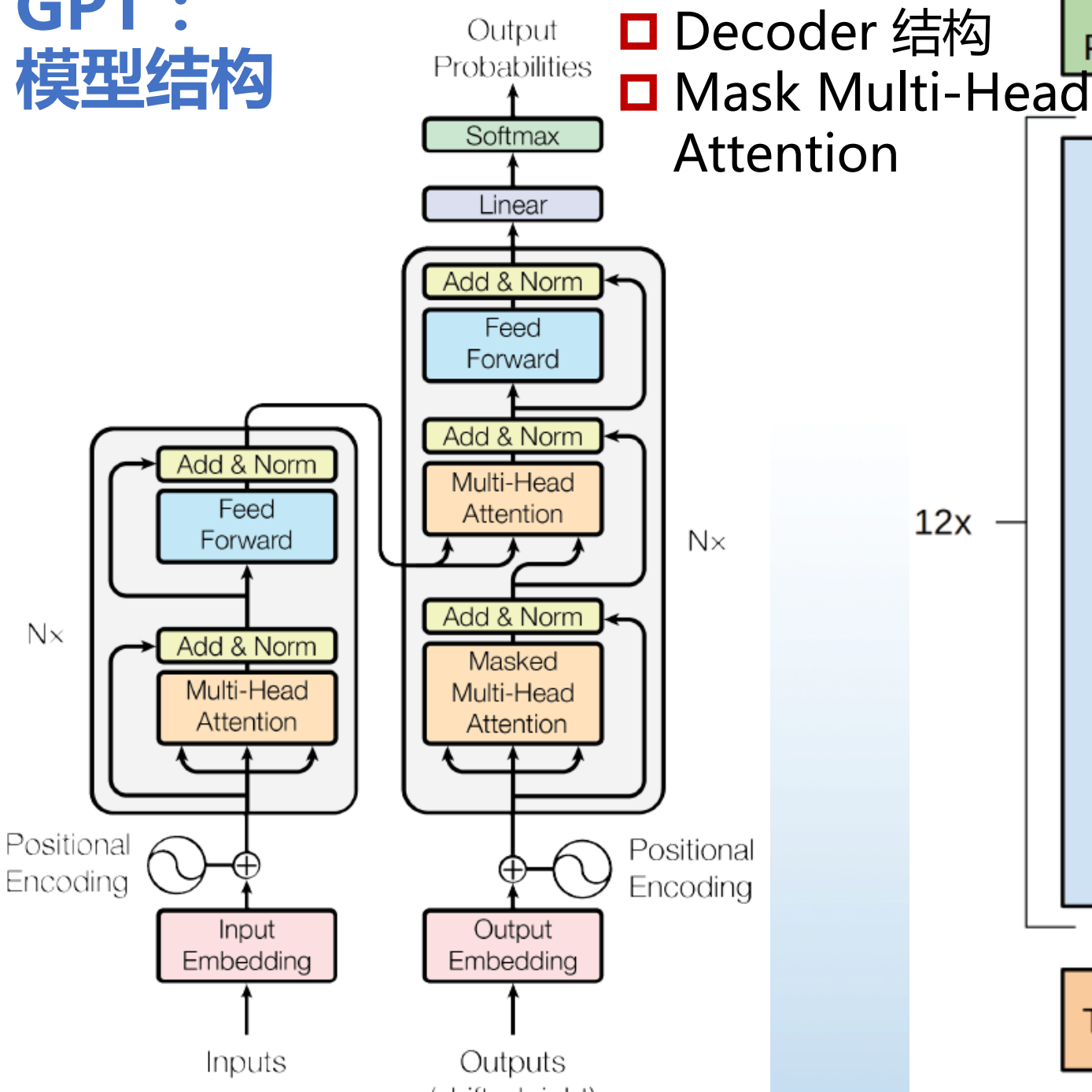
□ GPT: Generative Pre-Training

□ 目前已经公布论文的有 GPT-1、GPT-2、GPT-3 、GPT-3.5、 GPT-4

□ Transformer->GPT1->GPT2->GPT3->ChatGPT

模型	发布时间	参数量	预训练数据量
GPT	2018 年 6 月	1.17 亿	约 5GB
GPT-2	2019 年 2 月	15 亿	40GB
GPT-3	2020 年 5 月	1,750 亿	45TB

GPT : 模型结构



	GPT	Transformer
encoder	无	☐ Masked Multi-Attention ☐ Feed Forward
decoder	☐ Masked Multi-Attention ☐ Feed Forward	☐ Masked Multi-Attention ☐ Feed Forward ☐ Encoder-Decoder Attention
层数	12	6
注意力输出维数	768	512
Multi-heads	12	8
隐层维数	3072	2048
总参数	1.5亿	
目标任务	单序列文本的生成	机器翻译

GPT-1和Transformer 结构还有哪些差异？

GPT-1 采用的是单向的语言模型？

- ❑ GPT 中采用了 Masked Multi-Head Attention
- ❑ Masked Multi-Head Attention 只利用上文对当前位置的值预测
- ❑ GPT-1 被认为是单向的语言模型

GPT-1 中 Position Encoding 的操作有何不同？

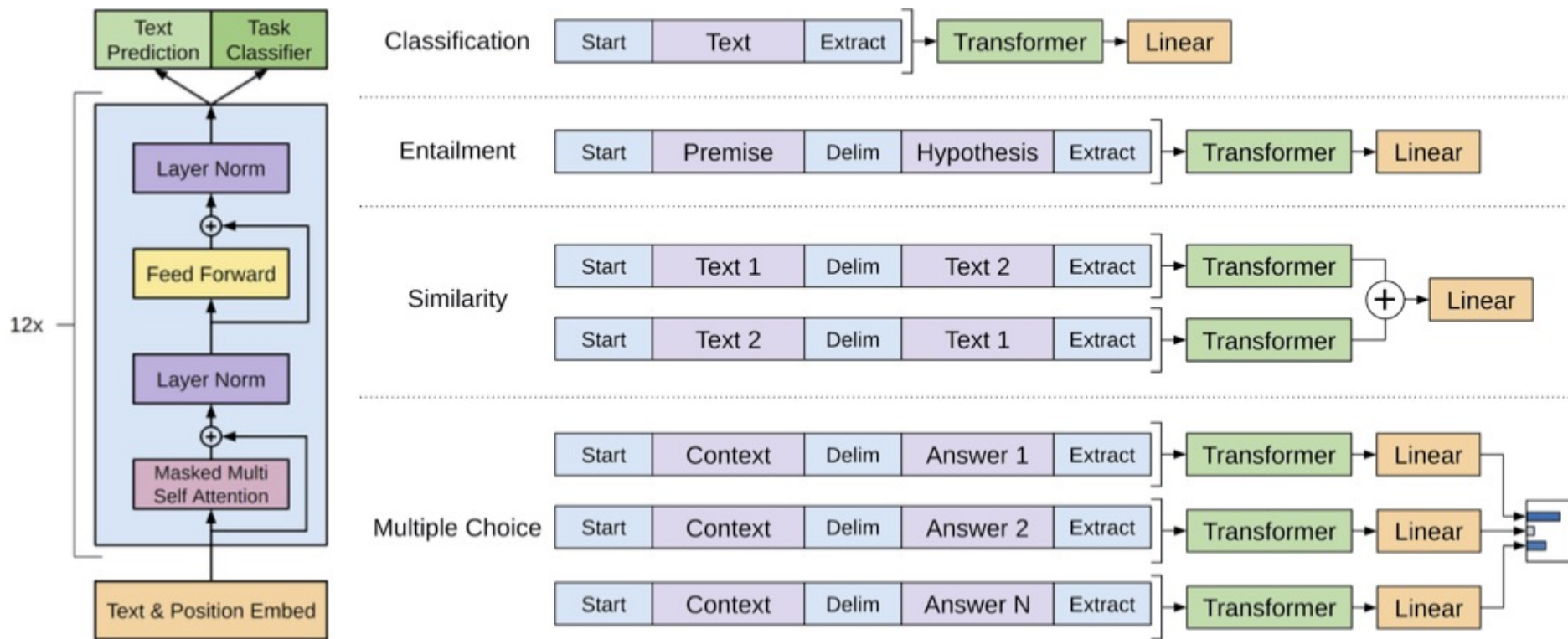
- ❑ Transformer 中，由于 Self-Attention 无法捕获文本的位置信息，因此需要对输入的词 Embedding 加入 Position Encoding，在 Transformer 中采用了 sin 和 cos 的计算方法
- ❑ GPT-1不再使用正弦和余弦的位置编码，而是采用与词向量相似的随机初始化，并在训练中进行更新

GPT : 模型结构

❑ GPT的整体结构是一个基于Transformer的单向语言模型

❑ GPT-1 的训练包含两阶段：

- ① GPT-1 模型的预训练过程，得到文本的语义向量；
- ② 在具体任务上 Fine-tuning，以解决具体的下游任务。



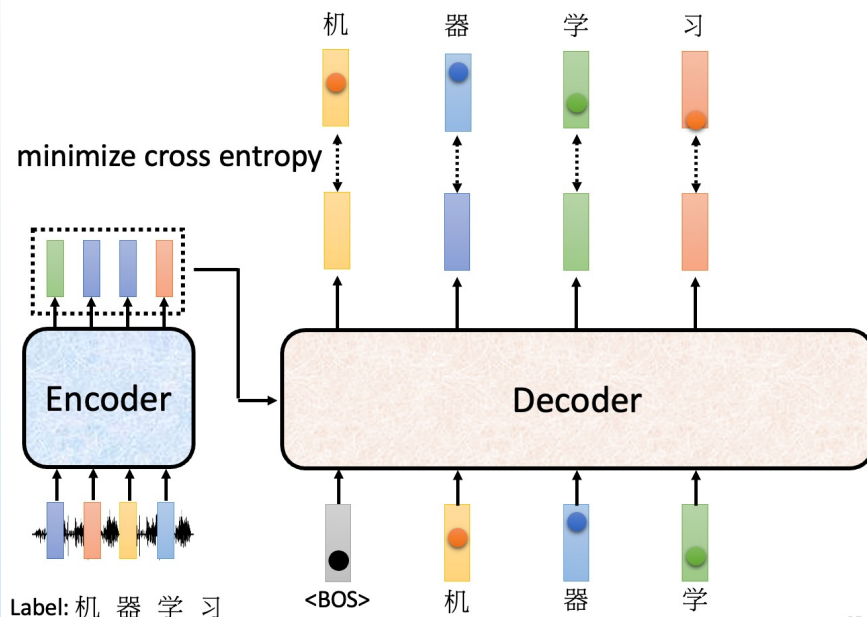
GPT：无监督预训练

□ 从左至右对输入文本进行建模(单向)

□ 给定文本序列 $x = x_1 \cdots x_n$ ，计算最大似然估计 L^{PT}

$$\mathcal{L}^{\text{PT}}(x) = \sum_i \log P(x_i | x_{i-k} \cdots x_{i-1}; \theta)$$

- 表示语言模型的窗口大小，即基于 k 个历史词 $x_{i-k} \cdots x_{i-1}$ 预测当前时刻的词 x_i ；
- θ 表示神经网络模型的参数，可使用随机梯度下降法优化该似然函数。



□ 从左至右对输入文本进行建模(单向)

- 对于长度为k的窗口词序列 $x' = x_{-k} \cdots x_{-1}$ ，通过以下方式计算建模概率P

$$\mathbf{h}^{[0]} = \mathbf{e}_{x'} \mathbf{W}^e + \mathbf{W}^p$$

$$\mathbf{h}^{[l]} = \text{Transformer-Block}(\mathbf{h}^{[l-1]}), \quad \forall l \in \{1, 2, \dots, L\}$$

$$P(x) = \text{Softmax}(\mathbf{h}^{[L]} \mathbf{W}^{e\top})$$

- $\mathbf{e}_{x'} \in \mathbb{R}^{k \times |\mathbb{V}|}$ 表示 x' 的独热向量表示
- $\mathbf{W}^e \in \mathbb{R}^{|\mathbb{V}| \times d}$ 表示词向量矩阵
- $\mathbf{W}^p \in \mathbb{R}^{n \times d}$ 表示位置向量矩阵(此处只截取窗口 x' 对应的位置向量)
- L表示Transformer的总层数

GPT：有监督任务精调

利用下游任务的有标注数据，对GPT模型进行精调

▣精调目的: 在通用语义表示的基础上，根据下游任务的特性进行领域适配，使之与下游任务的形式更加契合，以获得更好的下游任务应用效果

▣下游任务精调通常由有标注数据进行训练和优化的

▣假设下游任务的标注数据为C，其中每个样例的输入是 $x = x_1 \cdots x_n$ 构成的长度为n的文本序列，与之对应的标签为y

① 将文本序列输入预训练的GPT中，获取最后一层的最后一个词对应的隐含层输出 $\mathbf{h}_n^{[L]}$ ， $\mathbf{h}^{[l]} = \text{Transformer-Block}(\mathbf{h}^{[l-1]})$, $\forall l \in \{1, 2, \dots\}$,

② 将该隐含层输出通过一层全连接层变换，预测最终的标签

$$P(y|x_1 \cdots x_n) = \text{Softmax}(\mathbf{h}_n^{[L]} \mathbf{W}^y)$$

$\mathbf{W}^y \in \mathbb{R}^{d \times k}$ 表示全连接层权重，k表示标签个数

GPT：有监督任务精调

□利用下游任务的有标注数据，对GPT模型进行精调

□优化以下损失函数精调下游任务

$$\mathcal{L}^{\text{FT}}(\mathcal{C}) = \sum_{(x,y)} \log P(y|x_1 \cdots x_n)$$

□某些情况下，添加额外的预训练损失可以进一步提升性能

$$\mathcal{L}(\mathcal{C}) = \mathcal{L}^{\text{FT}}(\mathcal{C}) + \lambda \mathcal{L}^{\text{PT}}(\mathcal{C})$$

□缓解在下游任务精调的过程中出现灾难性遗忘问题

□在下游任务精调过程中，GPT的训练目标是优化下游任务数据上的效果，更强调特殊性

□势必会对预训练阶段学习的通用知识产生部分的覆盖或擦除，丢失一定的通用性

□结合下游任务精调损失和预训练任务损失，可以有效地缓解灾难性遗忘问题，在优化下游任务效果的同时保留一定的通用性

GPT：有监督任务精调

利用下游任务的有标注数据，对GPT模型进行精调

- 某些情况下，添加额外的预训练损失可以进一步提升性能

$$\mathcal{L}(C) = \mathcal{L}^{\text{FT}}(C) + \lambda \mathcal{L}^{\text{PT}}(C)$$

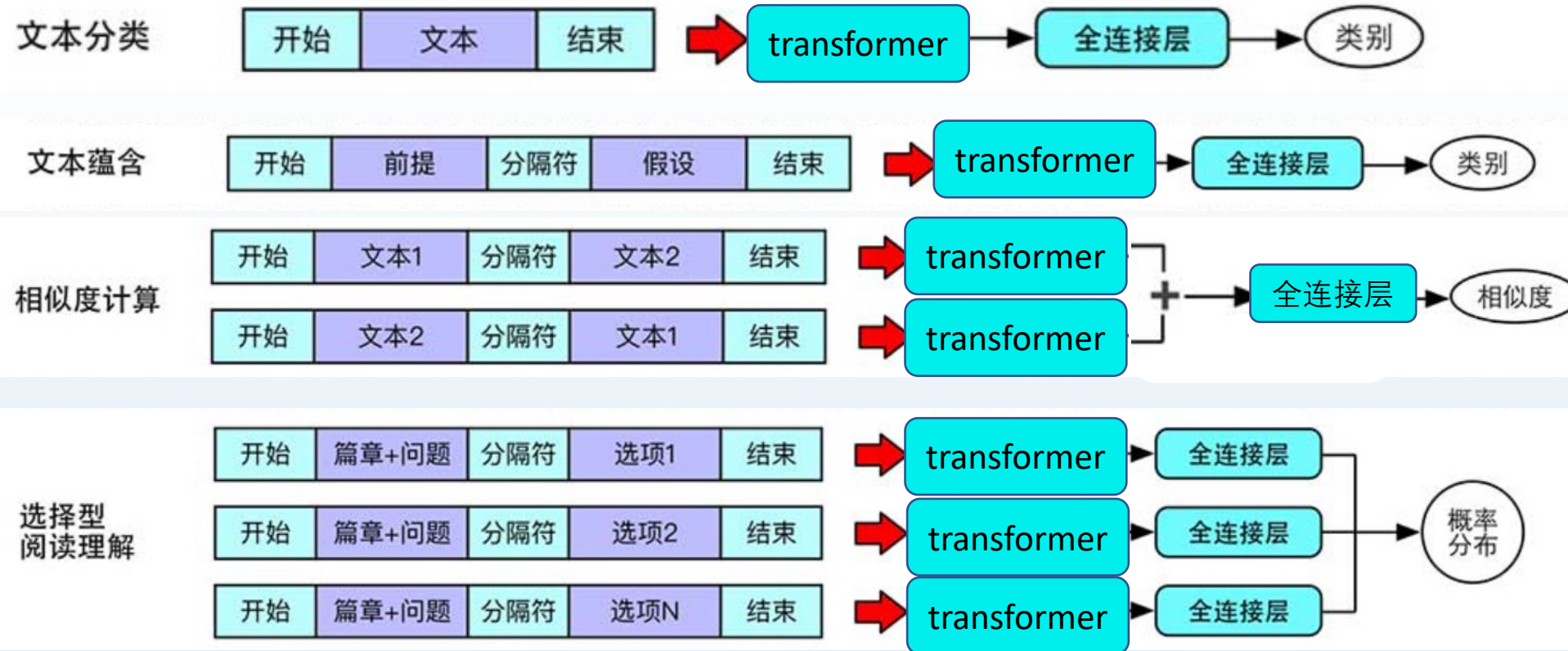
- $\lambda=0$ 时， $\mathcal{L}^{\text{PT}}$ 一项无效，只使用精调任务损失 $\mathcal{L}^{\text{FT}}$ 优化下游任务
- 当 $\lambda=1$ 时， $\mathcal{L}^{\text{PT}}$ 和 $\mathcal{L}^{\text{FT}}$ 具有相同的权重
- 在实际应用中，通常设置 $\lambda=0.5$
 - 在精调下游任务的过程中，主要目的是要优化有标注数据集的效果，即优化 $\mathcal{L}^{\text{FT}}$
 - $\mathcal{L}^{\text{PT}}$ 的引入主要是为了提升精调模型的通用性，其重要程度不及 $\mathcal{L}^{\text{FT}}$ ，设置 $\lambda=0.5$ 是一个较为合理的值

GPT：适配不同的下游任务

□根据任务特点，设置不同的输入输出形式

□GPT在下游任务精调的通用做法

□不同任务之间的输入形式各不相同，根据不同任务适配GPT的输入形式



GPT：适配不同的下游任务

① 单句文本分类

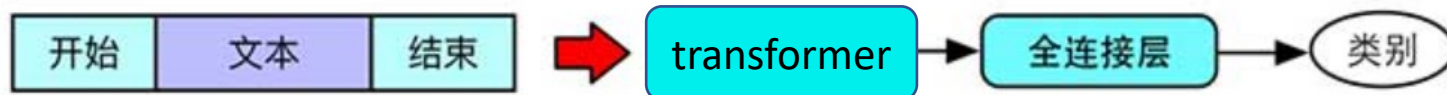
- 单句文本分类是最常见的自然语言处理任务之一
- 单句文本分类输入由单个文本构成，输出由对应的分类标签构成
- 假设输入为 $x = x_1 \cdots x_n$ ，单句文本分类的样例将通过如下形式输入GPT中

$\langle s \rangle x_1 x_2 \cdots x_n \langle e \rangle$

□ $\langle s \rangle$ 表示开始标记

□ $\langle e \rangle$ 表示结束标记

文本分类



GPT：适配不同的下游任务

② 文本蕴含

□ 文本蕴含的输入由两段文本构成，输出由分类标签构成，用于判断两段文本之间的蕴含关系

□ **注意**：文本蕴含中的前提(Premise)和假设(Hypothesis) 有序

□ 在所有样例中需要使用统一格式，两者顺序必须固定

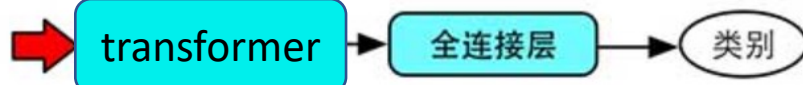
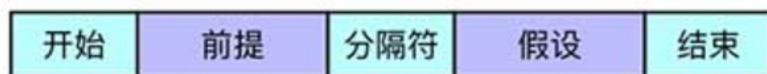
□ 前提在前或者假设在前

□ 假设文本蕴含的样例分别为 $x^{(1)} = x_1^{(1)} \dots x_n^{(1)}$ 和 $x^{(2)} = x_1^{(2)} \dots x_m^{(2)}$ ，其将通过如下形式输入GPT中

$$\langle s \rangle x_1^{(1)} \dots x_n^{(1)} \$ x_1^{(2)} \dots x_m^{(2)} \langle e \rangle$$

□ \$表示分隔标记，用于分隔两段文本，n和m分别表示 $x^{(1)}$ 和 $x^{(2)}$ 的长度

文本蕴含



GPT：适配不同的下游任务

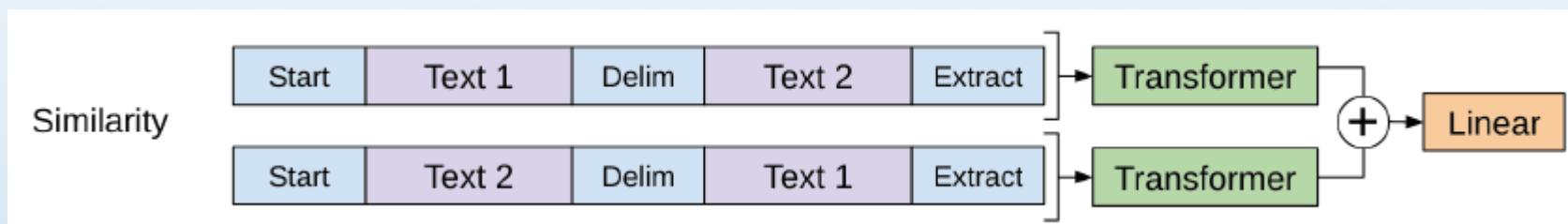
③ 相似度计算

- 相似度计算任务由两段文本构成
- 参与相似度计算的两段文本之间不存在顺序关系
- 假设相似度计算的样例分别为 $x^{(1)} = x_1^{(1)} \dots x_n^{(1)}$ 和 $x^{(2)} = x_1^{(2)} \dots x_m^{(2)}$ ，其将通过如下形式输入GPT中得到两个相应的隐含层表示

$\langle s \rangle x_1^{(1)} \dots x_n^{(1)} \$ x_1^{(2)} \dots x_m^{(2)} \langle e \rangle$

$\langle s \rangle x_1^{(2)} \dots x_m^{(2)} \$ x_1^{(1)} \dots x_n^{(1)} \langle e \rangle$

- 将这两个隐含层表示相加，并通过一个全连接层预测相似度



GPT：适配不同的下游任务

④ 选择型阅读理解(1/2)

□ 选择型阅读理解任务：让机器阅读一篇文章，从多个选项中选择出问题对应的正确选项

□ 输入：<篇章，问题，选项> 标签：正确选项编号

□ 假设篇章为 $p = p_1 p_2 \cdots p_n$ ，问题为 $q = q_1 q_2 \cdots q_m$ ，第 i 个选项为 $c^{(i)} = c_1^{(i)} \cdots c_n^{(i)}$ ， N 为选项个数，输入形式：

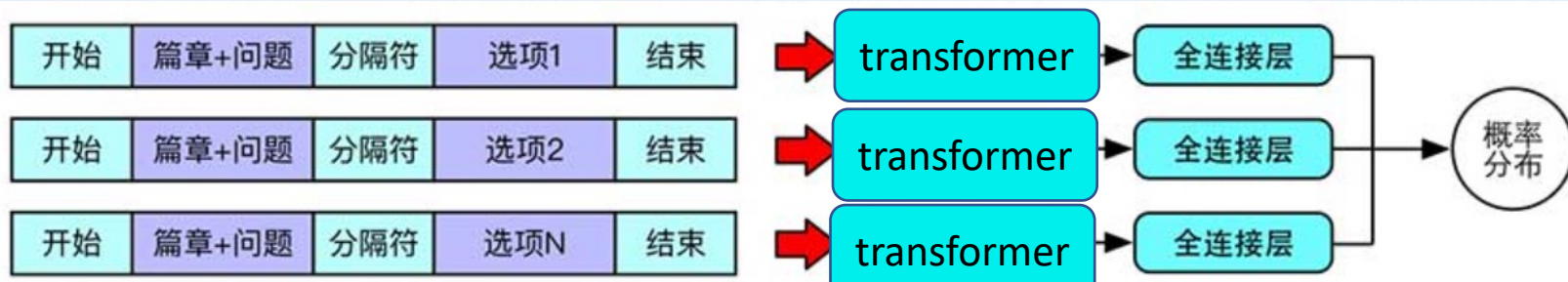
$\langle s \rangle p_1 p_2 \cdots p_n \$ q_1 q_2 \cdots q_m \$ c_1^{(1)} \cdots c_n^{(1)} \langle e \rangle$

$\langle s \rangle p_1 p_2 \cdots p_n \$ q_1 q_2 \cdots q_m \$ c_1^{(2)} \cdots c_n^{(2)} \langle e \rangle$

⋮

$\langle s \rangle p_1 p_2 \cdots p_n \$ q_1 q_2 \cdots q_m \$ c_1^{(N)} \cdots c_n^{(N)} \langle e \rangle$

选择型
阅读理解

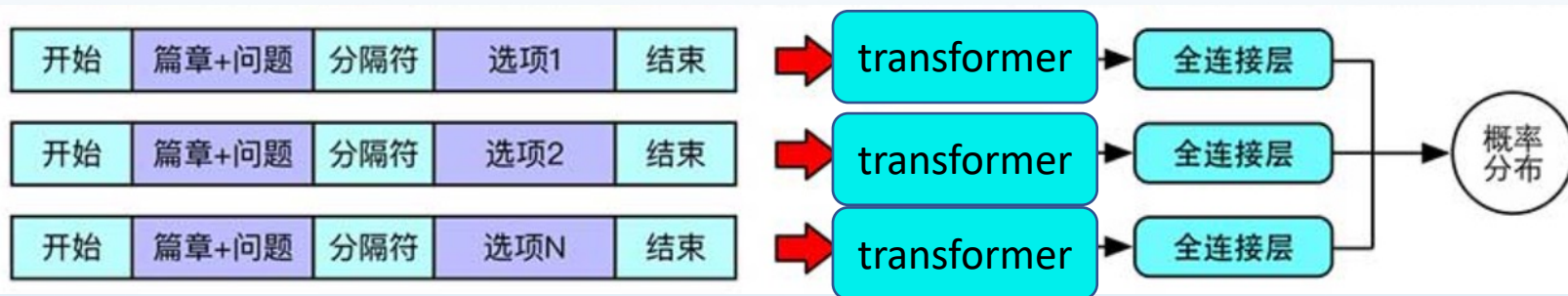


GPT：适配不同的下游任务

④ 选择型阅读理解(2/2)

- 通过GPT建模得到对应的隐含层表示，并通过全连接层得到每个选项的得分
- 将N个选项的得分拼接，通过Softmax函数得到归一化的概率(单选题)，并通过交叉熵损失函数学习

选择型
阅读理解



GPT-1 数据集

- ❑ BooksCorpus数据集，这个数据集包含7000 本没有发布的书籍
- ❑ 作者选这个数据集的原因有二：
 - ① 数据集拥有更长的上下文依赖关系，使得模型能学得更长期的依赖关系
 - ② 这些书籍因为没有发布，所以很难在下游数据集上见到，更能验证模型的泛化能力

GPT-1 性能

- ❑ 在有监督学习的12个任务中，GPT-1在9个任务上的表现超过了state-of-the-art的模型
- ❑ 在没有见过数据的零次学习(zero-shot) 任务中，GPT-1的模型要比基于LSTM的模型稳定，且随着训练次数的增加，GPT-1的性能也逐渐提升，表明GPT-1有非常强的泛化能力，能够用到和有监督任务无关的其它NLP任务中。
- ❑ GPT-1证明了transformer对学习词向量的强大能力，在GPT-1得到的词向量基础上进行下游任务的学习，能够让下游任务取得更好的泛化能力。对于下游任务的训练，GPT-1往往只需要简单的微调便能取得非常好的效果。

□ 代际关系总览



GPT系列模型对比总表					
维度	GPT-1 (2018)	GPT-2 (2019)	GPT-3 (2020)	GPT-3.5 (2022)	GPT-4 (2023)
参数量	117M	1.5B	175B	~200B (推测)	~1T (推测)
训练数据	BooksCorpus (5GB)	WebText (40GB)	45TB (混合语料)	扩展清洗数据+ 对话数据	多模态数据 (文本+图像)
核心架构	Transformer Decoder	Transformer Decoder	Transformer Decoder	改进版 Decoder	MoE (混合专家推测)
上下文窗口	512 tokens	1024 tokens	2048 tokens	4096 tokens	32K tokens (32k版本)
学习范式	预训练+微调	零样本 (Zero-shot)	小样本/上下文学习	RLHF+指令微调	多模态指令跟随
标志能力	基础文本生成	多任务零样本生成	复杂推理/代码生成	流畅对话/指令理解	多模态理解/高精度推理
最大突破	验证 Transformer 潜力	规模化零样本能力	超大规模上下文学习	对话优化	多模态与安全对齐
开源情况	开源	开源 (部分)	闭源 (API)	闭源 (API)	闭源 (API)
典型应用	文本补全	无监督翻译/摘要	API生态/研究工具	ChatGPT免费版	ChatGPT Plus/企业应用

GPT-核心演进路径分析

GPT-1 → GPT-2 : 零样本能力的发现

▣ **技术继承** : 均基于 Transformer Decoder , 保留自回归生成架构。

▣ **关键突破** :

▣ 数据量扩大 8倍 (5GB → 40GB) , 参数扩大 12倍 (117M → 1.5B) 。

▣ 提出零样本学习 , 取消任务微调 , 直接通过提示 (Prompt) 激活多任务能力。

▣ **典型改进** :

▣ GPT-1需针对翻译任务微调 → GPT-2可直接输入 "Translate English to French: hello → bonjour" 完成翻译。

GPT-核心演进路径分析

GPT-2 → GPT-3 : 规模化的质变

□ 技术继承 :

延续零样本范式, 但参数量暴增 **100倍+** (1.5B → 175B)。

□ 关键突破 :

- 引入 **小样本学习 (Few-shot Learning)** : 提供3-5个示例即可适应新任务。

- 上下文窗口翻倍 (1024 → 2048 tokens) , 生成长文本能力显著提升。

□ 典型改进 :

- GPT-2生成代码易出错 → GPT-3可生成完整Python函数 (如排序算法)。

GPT-核心演进路径分析

❑ GPT-3 → GPT-3.5 : 对话专用优化

❑ 技术继承：

基于GPT-3架构，参数量相近(推测约200B)。

❑ 关键突破：

❑ **RLHF(人类反馈强化学习)**：通过人类偏好数据,优化对话安全性和流畅性。

❑ **指令微调**：适应更自然的用户指令(如“解释量子力学”)。

❑ 典型应用：

❑ 作为 **ChatGPT (免费版)** 的底层模型。

GPT-核心演进路径分析

❑ GPT-3.5 → GPT-4：多模态与可靠性跃迁

❑ 技术继承：

保留指令跟随能力，但架构可能升级为 **混合专家（MoE）**。

❑ 关键突破：

- ❑ **多模态输入**：支持图像理解（如描述图表内容）。

- ❑ **长上下文**：32k tokens版本可处理超长文档（如整本小说）。

- ❑ **安全对齐**：减少有害/偏见输出（比GPT-3.5降低82%违规内容）。

❑ 典型应用：

- ❑ **驱动 ChatGPT Plus 和微软Copilot。**

□技术主线：

- 规模驱动**：参数量和数据持续扩大（GPT-1的117M → GPT-4的~1T）。
- 范式演进**：从微调（GPT-1）→ 零样本（GPT-2）→ 上下文学习（GPT-3）→ 多模态（GPT-4）。

□产品化转折点：

- GPT-3.5** 是对话优化的分水岭，**GPT-4** 实现从文本到多模态的跨越。

□核心局限：

- 越先进的模型越依赖闭源和API，开源生态停滞（GPT-2是最后开源的主要版本）。

GPT-代际关系对比表

关系维度	GPT-1 → GPT-2	GPT-2 → GPT-3	GPT-3 → GPT-3.5	GPT-3.5 → GPT-4
核心目标	验证零样本潜力	探索超大规模效应	优化对话交互	实现多模态与可靠性
技术连续性	同架构，规模扩大	同架构，规模爆炸	增加RLHF微调	架构可能升级（MoE）
数据变化	数据量×8	数据量×1000+	加入人类反馈数据	加入多模态数据
能力跃迁	多任务零样本生成	复杂推理/代码生成	安全流畅的对话	图像理解/长上下文处理
应用场景扩展	实验性文本生成	API生态开发	ChatGPT产品化	企业级工具集成

目录

CONTENTS

1

概述

2

自回归预训练模型代表：GPT

3

自编码预训练模型代表：BERT

4

预训练语言模型的应用

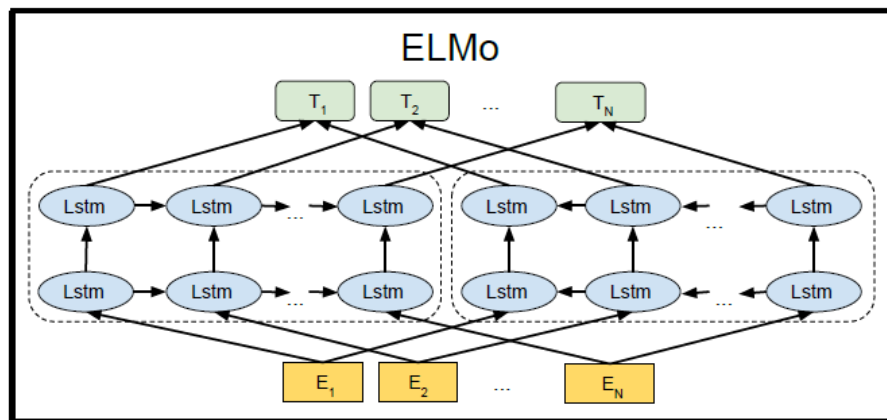
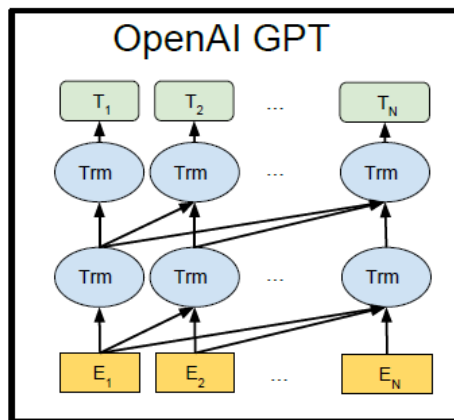
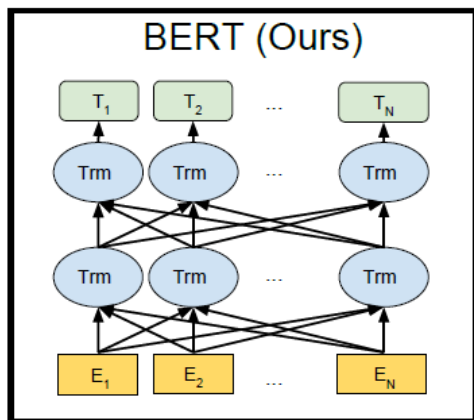
5

深入理解BERT

- ❑ **BERT: Bidirectional Encoder Representations from Transformers** (NAACL 2019 Best Paper)
 - ❑ 2018年10月由Google AI研究院提出的一种预训练模型
 - ❑ 提出了一种**双向**预训练语言模型方法
 - ❑ 利用大规模自由文本训练两个无监督预训练任务
 - ❑ BERT在众多NLP任务中获得了显著性能提升
 - ❑ 进一步强调了使用通用预训练取代繁杂的任务特定的模型设计

□ GPT/BERT/ELMo之间的对比

- GPT: **单向**从左至右的Transformer语言模型
- ELMo: 将**独立的**前向和后向的LSTM语言模型拼接所得
- BERT: **双向** Transformer语言模型



预训练+精调

预训练

BERT : 模型结构

整体结构

由深层Transformer模型构成

base : 12层, 参数量110M

large : 24层, 参数量330M

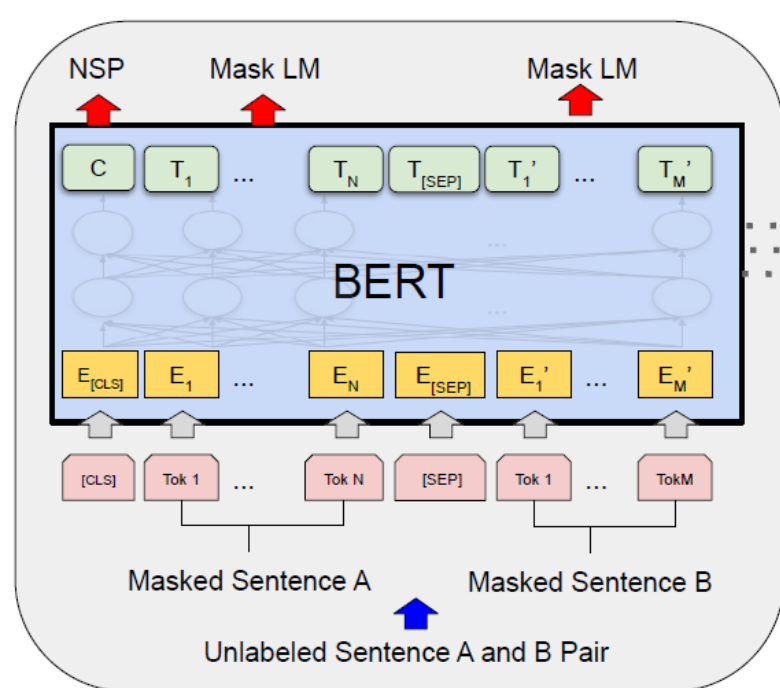
预训练任务

掩码语言模型(MLM)

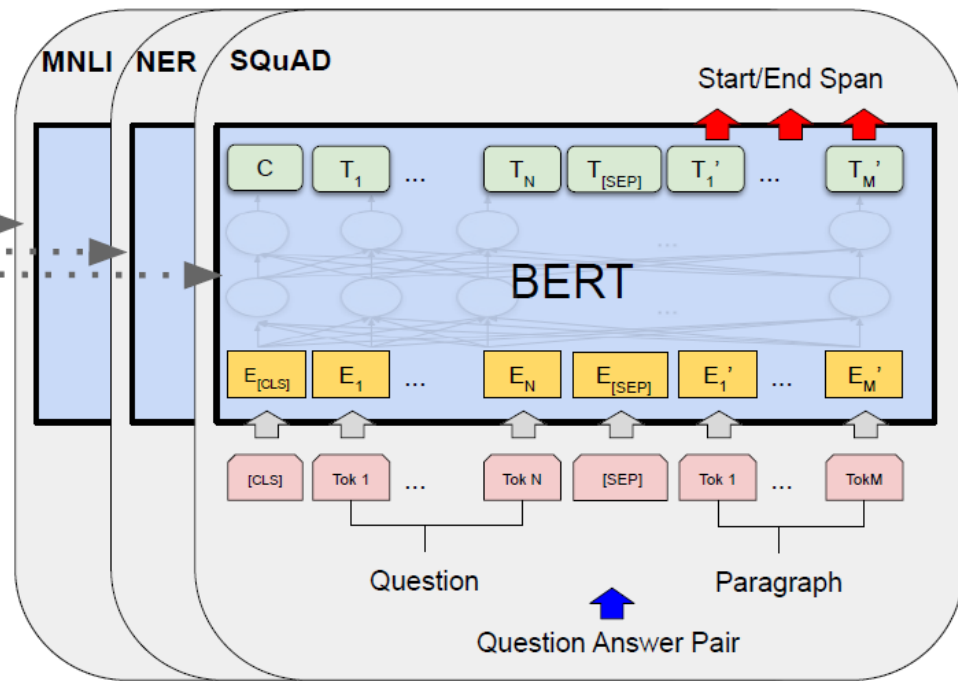
Masked Language Model

下一个句子预测(NSP)

Next Sentence Prediction



Pre-training



Fine-Tuning

BERT：输入表示

□ BERT的输入表示由三部分组成

- 词向量：通过词向量矩阵将输入文本转换为实值向量表示
- 块向量：编码当前词属于哪一个块
 - 注意 [CLS]位（输入序列中的第一个标记）和第一个块结尾处的[SEP]位（用于分隔不同块的标记）的块编码均为0
- 位置向量：编码当前词的绝对位置

原始输入	[CLS]	天	气	晴	朗	[SEP]	我	喜	欢	[SEP]
词向量	$v_{[CLS]}$	$v_{\text{天}}$	$v_{\text{气}}$	$v_{\text{晴}}$	$v_{\text{朗}}$	$v_{[SEP]}$	$v_{\text{我}}$	$v_{\text{喜}}$	$v_{\text{欢}}$	$v_{[SEP]}$
	+	+	+	+	+	+	+	+	+	+
块向量	v_A	v_A	v_A	v_A	v_A	v_A	v_B	v_B	v_B	v_B
	+	+	+	+	+	+	+	+	+	+
位置向量	v_0	v_1	v_2	v_3	v_4	v_5	v_6	v_7	v_8	v_9

BERT：基本预训练任务

▣预训练任务1：Masked Language Model (MLM)

- ▣将输入序列中的部分token进行掩码，并且要求模型将它们进行还原
- ▣在BERT中，会将**15%**的输入文本进行mask
 - ▣以 80% 的概率替换为 [MASK] 标记
 - ▣以 10% 的概率替换为词表中的任意一个随机词
 - ▣以 10% 的概率保持原词不变，即不替换

原文本	
80% 概率替换为 [MASK]	The man went to the [MASK] to buy some milk.
10% 概率替换为随机词	The man went to the apple to buy some milk.
10% 概率保持原样	The man went to the store to buy some milk.

BERT : 基本预训练任务

预训练任务1 : Masked Language Model (MLM)

输入层 $X = [\text{CLS}] x'_1 x'_2 \cdots x'_n [\text{SEP}]$

$v = \text{InputRepresentation}(X)$

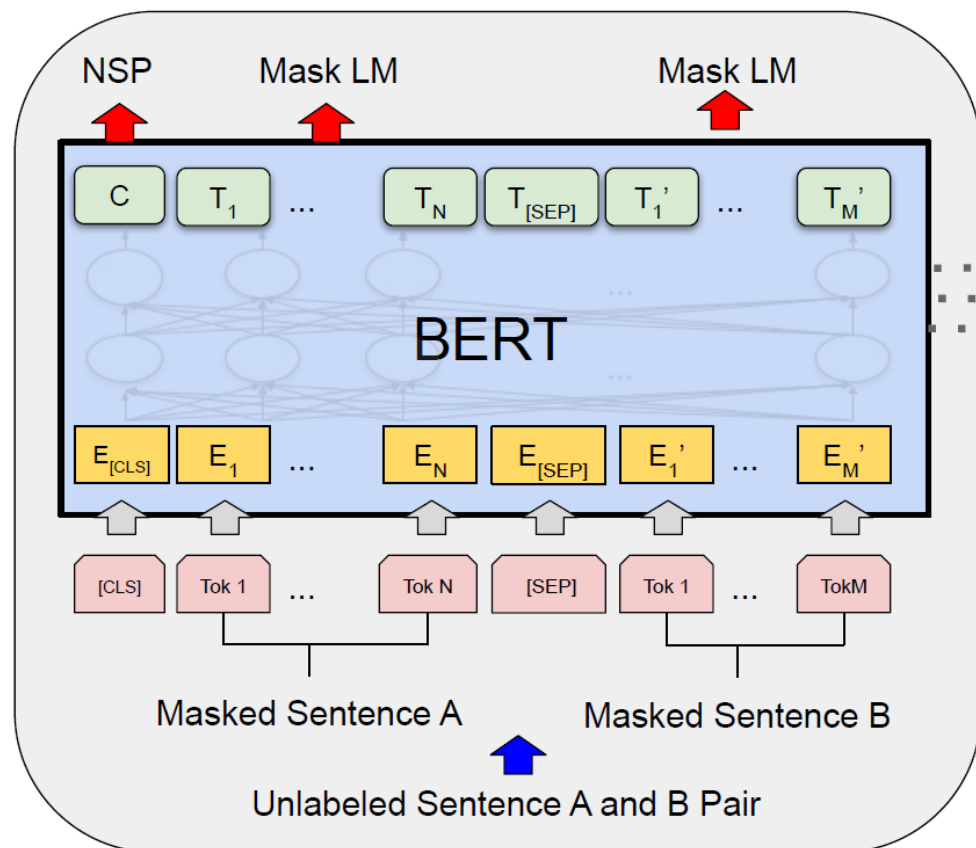
BERT编码层

$h = \text{Transformer}(v)$

输出层

$P_i = \text{Softmax}(h_i^m W^{t\top} + b^o)$

p_i 与标签 y_i 计算交叉熵损失，
学习模型参数



BERT：基本预训练任务

□预训练任务1：Masked Language Model (MLM)

□输入层

□如果输入文本的长度 n 小于BERT的最大序列长度 N ，需要将**补齐标记**[PAD]拼接在输入文本后，直至达到BERT的最大序列长度 N

□例如 假设BERT的最大序列长度 $N=10$ ，而输入序列长度为7（两个特殊标记加上 x_1 至 x_5 ），需要在输入序列后方添加3个[PAD]补齐标记，[CLS]
 $x_1 x_2 x_3 x_4 x_5$ [SEP] [PAD] [PAD] [PAD]

□输入序列 X 的长度**大于**BERT的最大序列长度 N ，需要对输入序列 X **截断**至BERT的最大序列长度 N 。

□例如 假设BERT的最大序列长度 $N=5$ ，而输入序列长度为7（两个特殊标记加上 x_1 至 x_5 ），需要对序列截断，使有效序列（输入序列中去除2个特殊标记）长度变为3。[CLS] $x_1 x_2 x_3$ [SEP]。

BERT：基本预训练任务

▣预训练任务2：Next Sentence Prediction (NSP)

▣学习两段文本之间的关系（上下文信息）

▣预测 *Sentence B* 是否是 *Sentence A* 的下一个句子

▣正样本：文本中相邻的两个句子“句子 A”和“句子 B”，构成“下一个句子”关系

▣负样本：将“句子 B”替换为语料库中任意一个句子，构成“非下一个句子”关系

	正样本	负样本
第一段文本	The man went to the store.	The man went to the store.
第二段文本	He bought a gallon of milk.	Penguins are flightless.

BERT : 基本预训练任务

□预训练任务2 : Next Sentence Prediction (NSP)

□输入层

$$X = [\text{CLS}] x_1^{(1)} x_2^{(1)} \cdots x_n^{(1)} [\text{SEP}] x_1^{(2)} x_2^{(2)} \cdots x_m^{(2)} [\text{SEP}]$$

$$v = \text{InputRepresentation}(X)$$

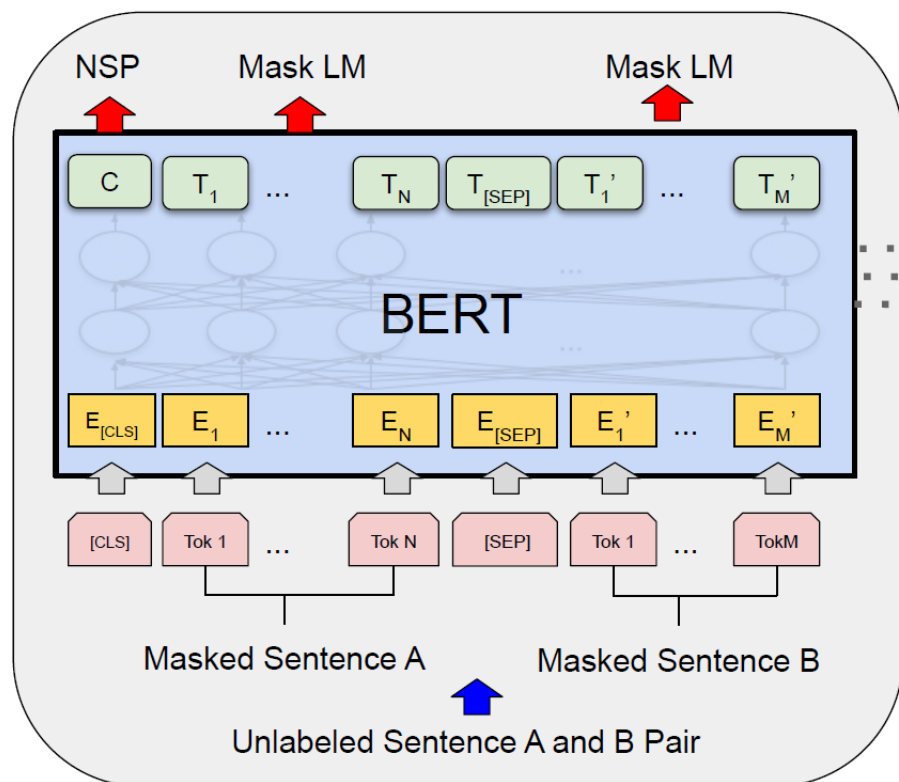
□BERT编码层

$$h = \text{Transformer}(v)$$

□输出层

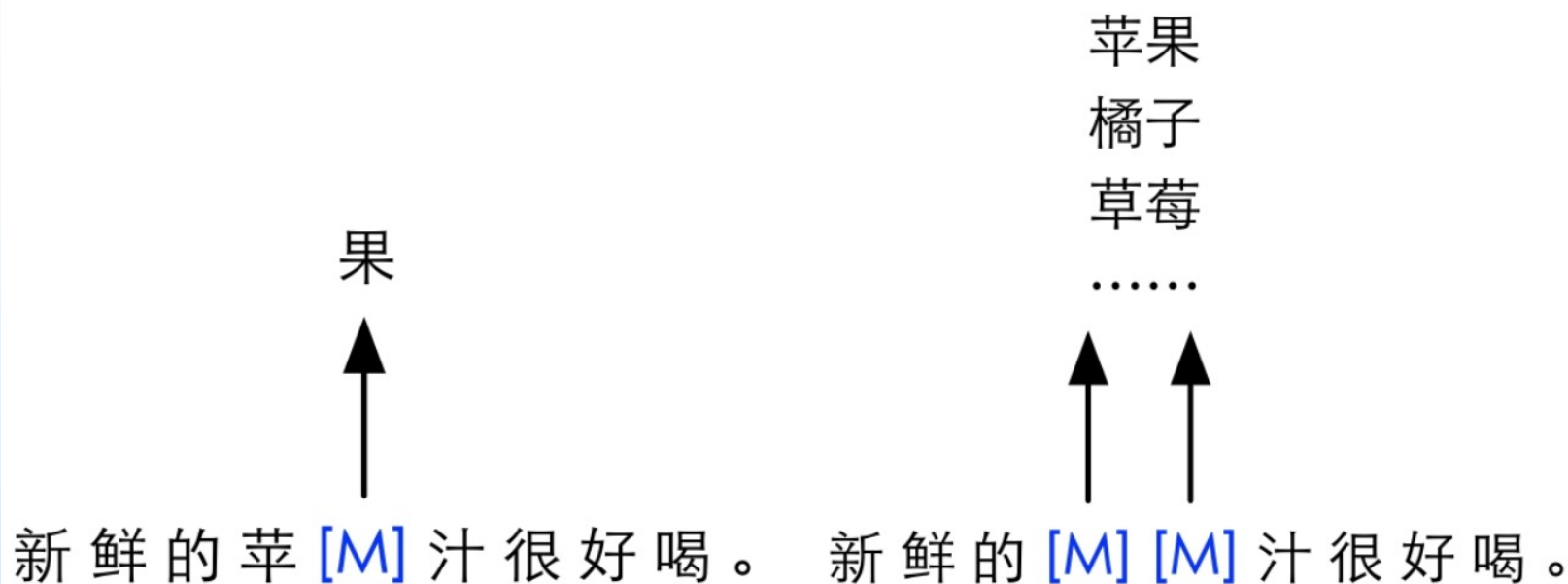
$$P = \text{Softmax}(h_0 W^p + b^o)$$

p与分类标签y计算交叉熵损失，学习模型参数



BERT：更多预训练任务

- ❑ 在MLM任务中，最小的掩码单位是WordPiece子词(中文：字)
- ❑ 问题：WordPiece子词信息泄露问题
 - ❑ 当一个整词的部分WordPiece子词被掩码时，仅依靠未被掩码的部分可较为容易地预测出掩码位置对应的原WordPiece子词



(a) 以字为掩码单位

(b) 以词为掩码单位

BERT：更多预训练任务

□整词掩码(Whole Word Masking)

- MLM：随机选取一定比例的WordPiece子词
- WWM：随机选取一定比例的**整词**，属于同一个整词的WordPiece子词均被掩码
 - 总掩码数量不变，变动的是掩码位置的选取
 - 子词的掩码方式并不统一，并不是采用一样的掩码方式（三选一）
 - 子词各自的掩码方式受概率控制

原始MLM掩码和整词掩码的对比示例

原始句子	the man jumped up , put his basket on phil ##am ##mon ' s head
原始掩码输入	[M] man [M] up , put his [M] on phil [M] ##mon ' s head
整词掩码输入	the man [M] up , put his basket on [M] [M] [M] ' s head

中文整词掩码的示例

原始句子	使用语言模型来预测下一个词的概率。
中文分词	使用 语言 模型 来 预测 下一个 词 的 概率 。
原始掩码输入	使 [M] 语 言 [M] 型 来 [M] 测 下一 [M] 词 的 概率 。
整词掩码输入	使用 语言 [M] [M] 来 [M] [M] 下一个 词 的 概率 。

BERT：更多预训练任务

□ N-gram掩码 (N-gram Masking)

- 对一个连续的N-gram单元进行掩码，进一步增加MLM任务的难度
- 难度：N-gram Masking > Whole Word Masking > MLM

We went to the store to buy some fruits.

→ We went to [M] store to [M] some [M].

Original
MLM

→ We went to [M] [M] [M] buy some fruits.

N-gram
Masking

□三种掩码策略的联系与区别

	MLM	WWM	NM
最小掩码单位（英文）	WordPiece 子词	WordPiece 子词	WordPiece 子词
最小掩码单位（中文）	字	字	字
最大掩码单位（英文）	WordPiece 子词	词	多个子词
最大掩码单位（中文）	字	词	多个字

BERT、GPT、ELMo和Word2vec之间的对比

对比项目	BERT	GPT	ELMo	Word2vec
基本结构	Transformer	Transformer	Bi-LSTM	MLP
训练任务	MLM/NSP	LM	BiLM	Skip-gram 或 CBOW
建模方向	双向	单向	双向	双向
静态/动态	动态	动态	动态	静态
参数量	大	大	中	小
解码速度	慢	慢	中	快
常规应用模式	预训练 + 精调	预训练 + 精调	词特征提取	词向量

目录

CONTENTS

1

概述

2

自回归预训练模型代表：GPT

3

自编码预训练模型代表：BERT

4

预训练语言模型的应用

5

深入理解BERT

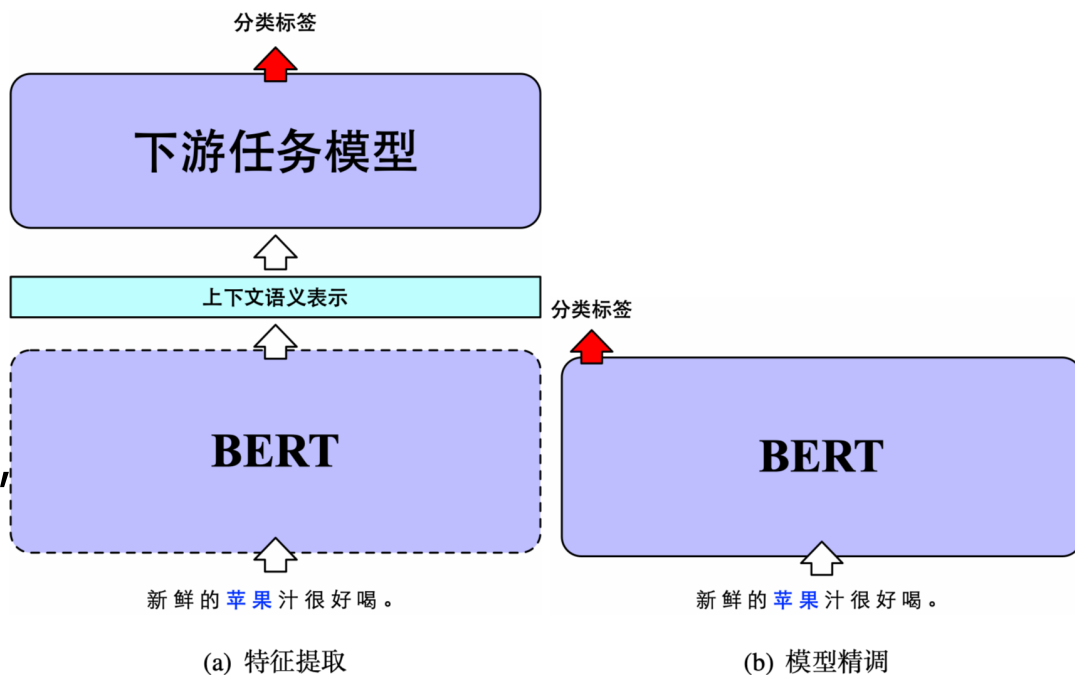
特征提取和模型精调

特征提取

- 仅利用BERT提取输入文本特征，生成对应的上下文语义表示
- BERT本身不参与目标任务的训练，即BERT部分只进行解码（无梯度回传）

模型精调

- 利用BERT作为下游任务模型基底，生成文本对应的上下文语义表示
- 参与下游任务的训练，即在下游任务学习过程中，BERT对自身参数进行更新



通常使用“模型精调”的方法，因其效果更佳

预训练语言模型应用

□特征提取

- 优点：特征提取方法与传统的词向量技术类似，使用起来相对简单，不参与下游任务的训练，在训练效率上相对较高
- 缺点：不参与下游任务的训练，本身无法根据下游任务进行适配，更多依赖于下游任务模型的设计，进一步加大了建模难度

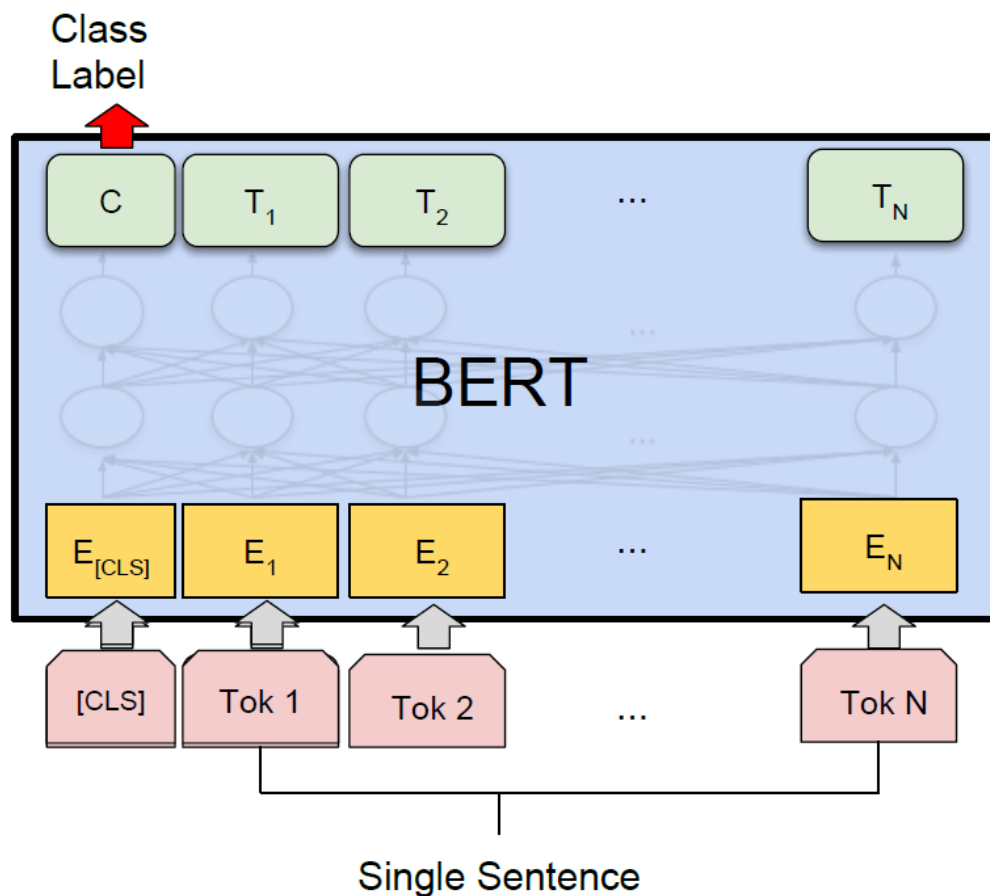
□模型精调

- 优点：能够充分利用预训练语言模型庞大的参数量学习更多的下游任务知识，使预训练语言模型与下游任务数据更加适配
- 缺点：预训练语言模型参与下游任务的训练，需要更大的参数存储量以存储模型更新所需的梯度，在模型训练效率上存在一定的劣势

预训练语言模型应用

单句文本分类 (Single Sentence Classification)

- 最常见的自然语言处理任务，需要将输入文本分成不同类别
- 例如：将影评文本输入到分类模型中，将其分成“褒义”和“贬义”类别



□ 单句文本分类 (Single Sentence Classification)

□ 输入层

$$X = [\text{CLS}] x_1 x_2 \cdots x_n [\text{SEP}]$$

$$v = \text{InputRepresentation}(X)$$

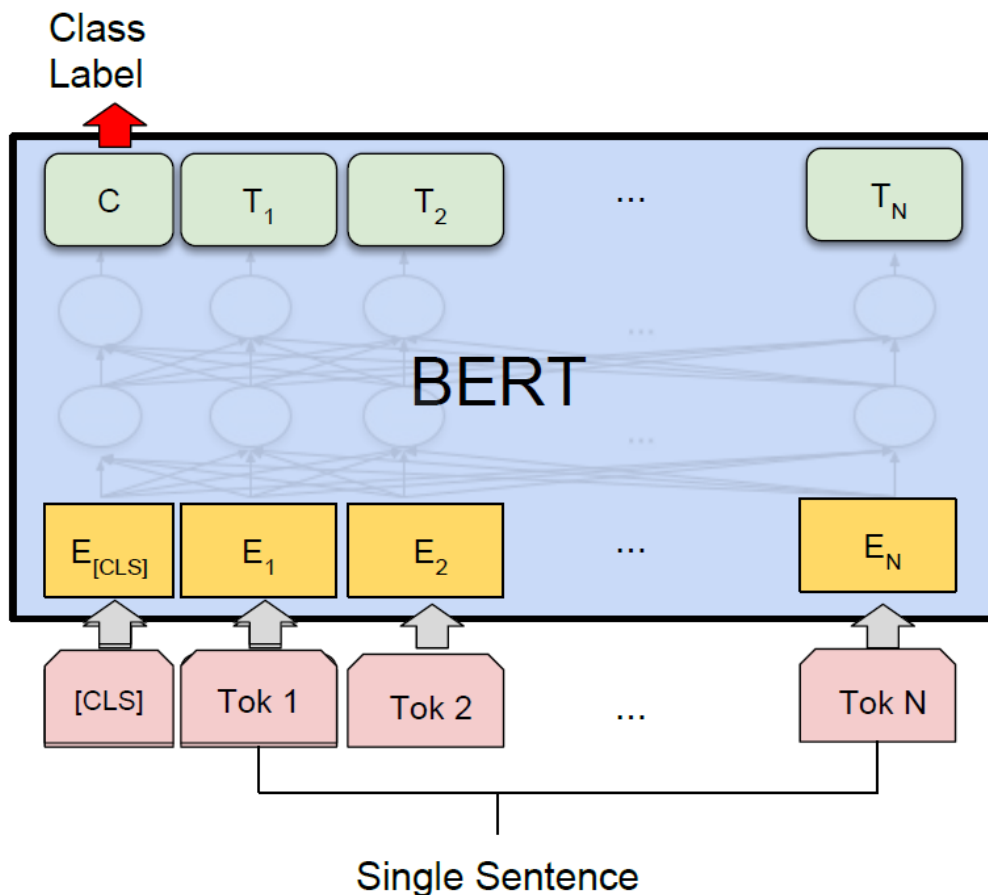
□ BERT编码层

$$h = \text{BERT}(v)$$

□ 分类输出层

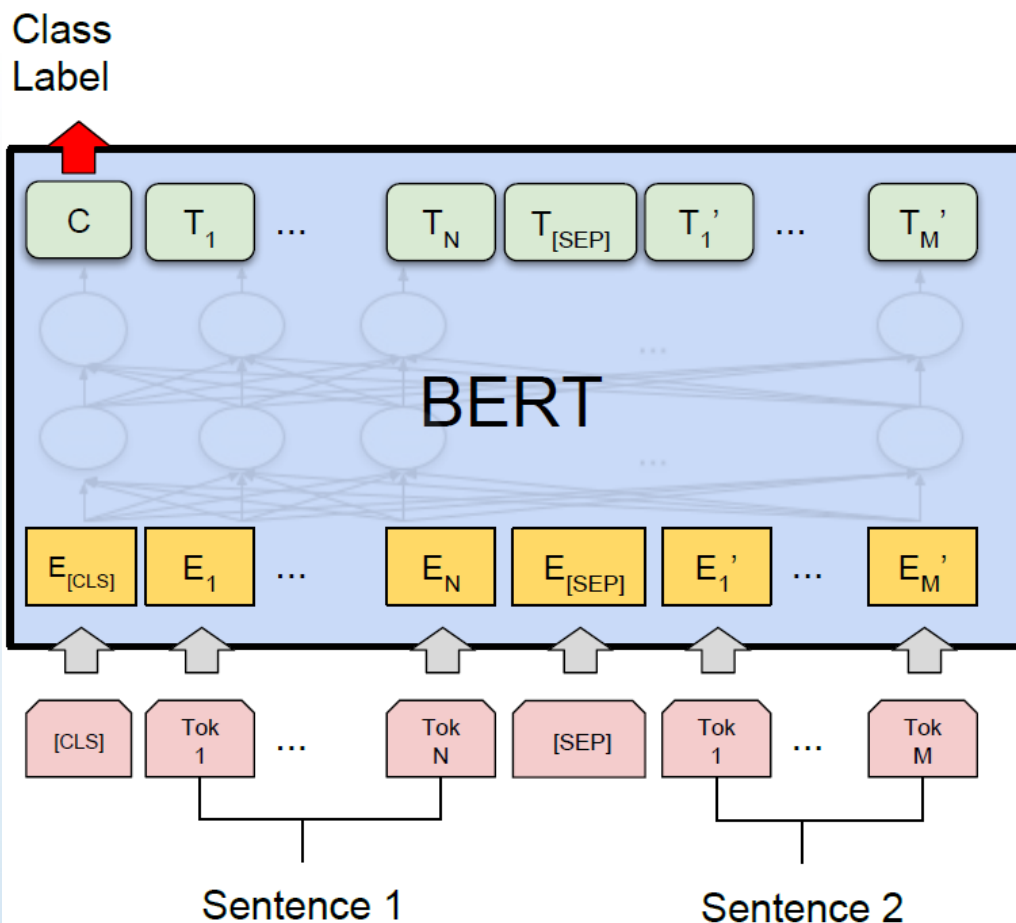
$$P = \text{Softmax}(h_0 W^0 + b^0)$$

p与真实分类标签y计算交叉熵损失，学习模型参数



□ 句对文本分类 (Sentence Pair Classification)

- 与单句文本分类任务类似，需要将一对文本分成不同类别
- 例如：文本蕴含任务中，将句对分成“蕴含”或者“冲突”类别



预训练语言模型应用

句对文本分类 (Sentence Pair Classification)

输入层 $X = [\text{CLS}] x_1^{(1)} x_2^{(1)} \dots x_n^{(1)} [\text{SEP}] x_1^{(2)} x_2^{(2)} \dots x_m^{(2)} [\text{SEP}]$

$v = \text{InputRepresentation}(X)$

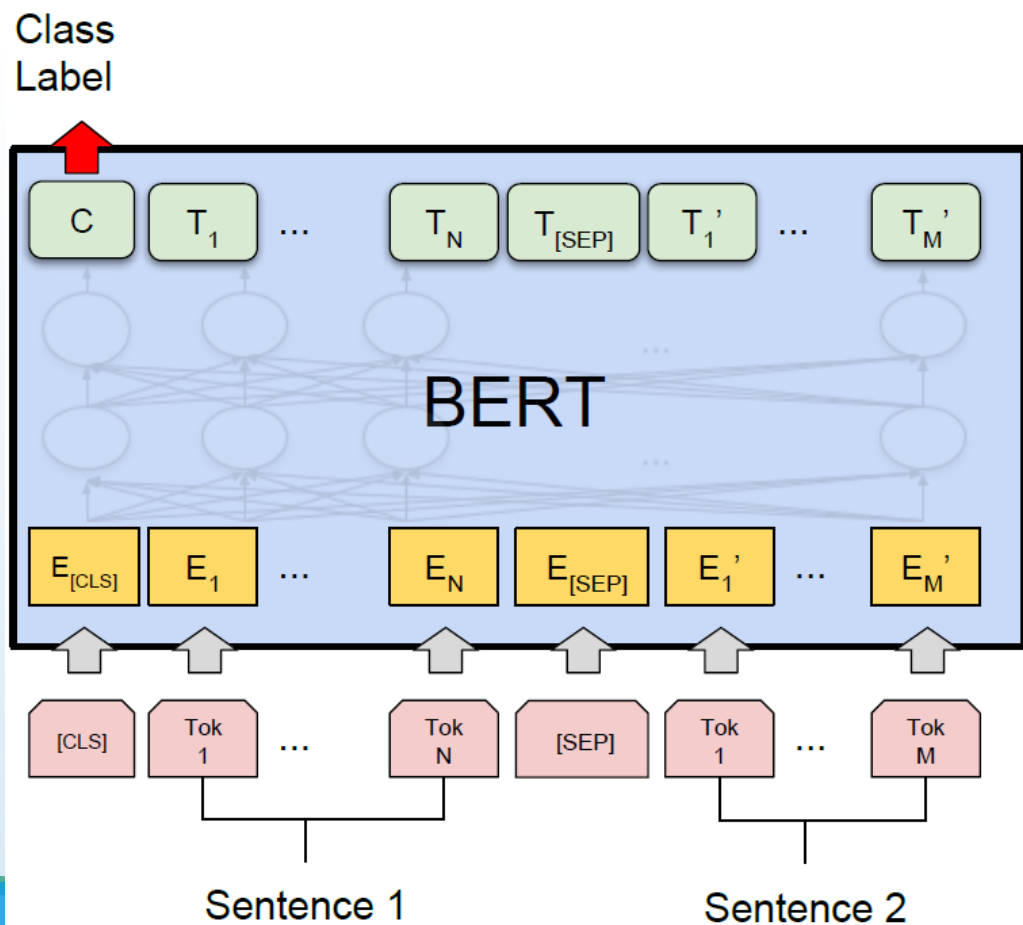
BERT编码层

$$h = \text{BERT}(v)$$

分类输出层

$$P = \text{Softmax}(h_0 W^0 + b^0)$$

p与分类标签y计算交叉熵损失，学习模型参数



□ 阅读理解

□ (Reading Comprehension)

□ 以抽取式阅读理解为例进行说明，要求机器在阅读篇章和问题后给出相应的答案，而答案要求是从篇章中抽取出的一个文本片段 (Span)

【篇章】

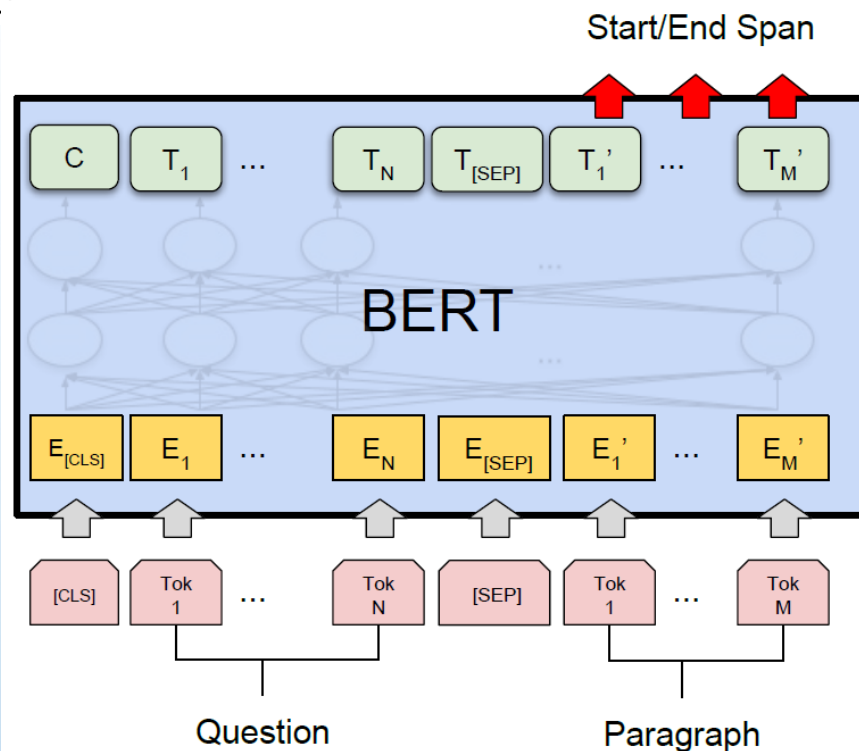
哈尔滨工业大学（简称哈工大）隶属于工业和信息化部，以理工为主，理工管文经法艺等多学科协调发展，拥有哈尔滨、威海、深圳三个校区。学校始建于 1920 年，1951 年被确定为全国学习国外高等教育办学模式的两所样板大学之一，1954 年进入国家首批重点建设的 6 所高校行列，曾被誉为工程师的摇篮。学校于 1996 年进入国家“211 工程”首批重点建设高校，1999 年被确定为国家首批“985 工程”重点建设的 9 所大学之一，2000 年与同根同源的哈尔滨建筑大学合并组建新的哈工大，2017 年入选“双一流”建设 A 类高校名单。

【问题】

哈尔滨工业大学哪一年入选国家首批“985 工程”？

【答案】

1999 年



预训练语言模型应用

阅读理解 (Reading Comprehension)

输入层 $X = [\text{CLS}] q_1 q_2 \cdots q_n [\text{SEP}] p_1 p_2 \cdots p_m [\text{SEP}]$

$v = \text{InputRepresentation}(X)$

BERT编

$h = \text{BERT}(v)$

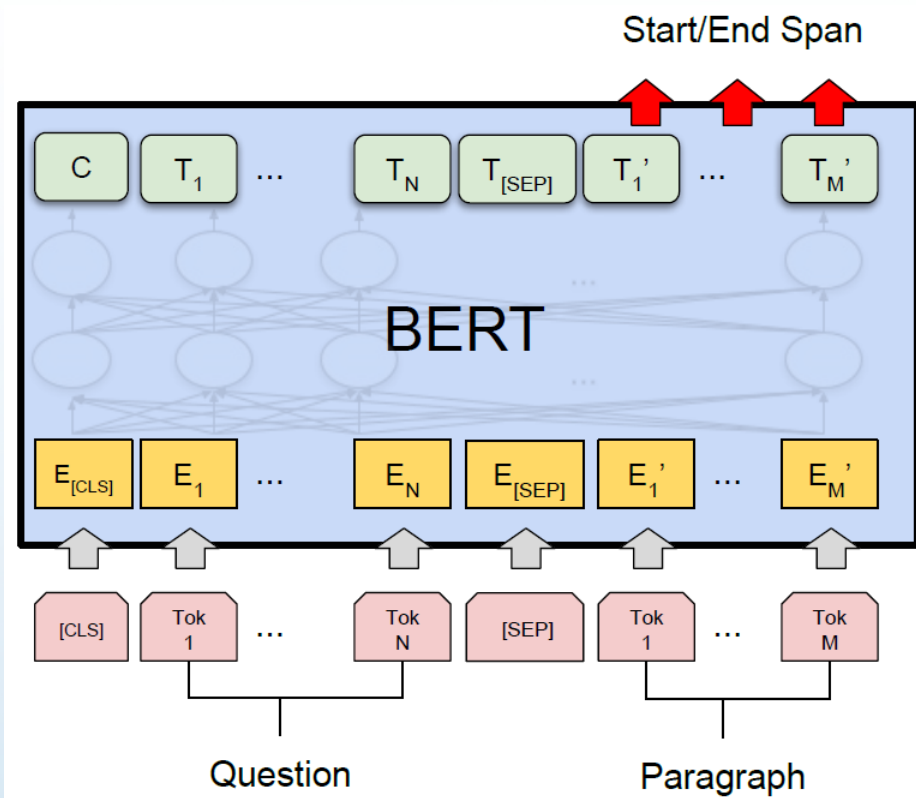
答案输出层

$P^s = \text{Softmax}(hW^s + b^s)$

$P^e = \text{Softmax}(hW^e + b^e)$

与起始位置和终止位置的交叉熵求平均，得最终损失

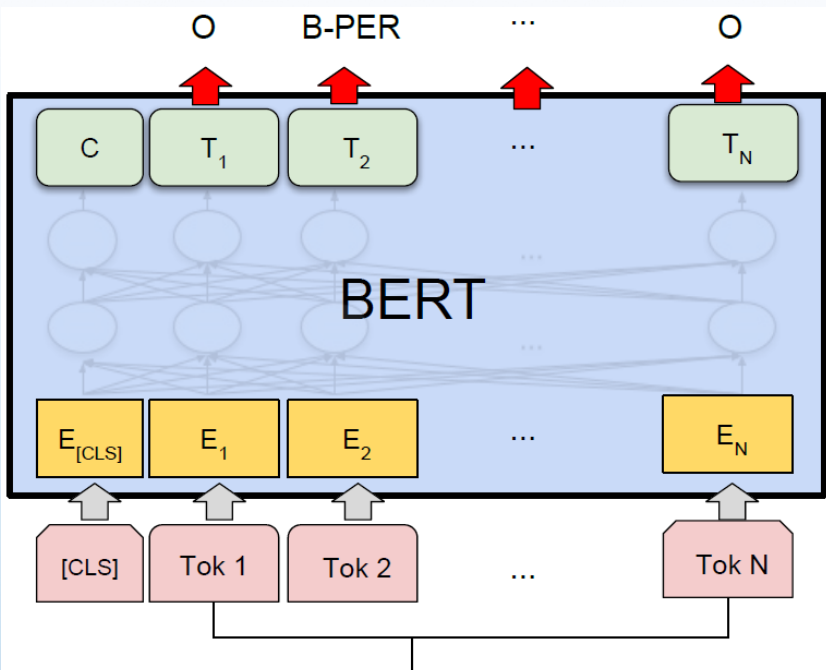
$$\mathcal{L} = \frac{1}{2}(\mathcal{L}^s + \mathcal{L}^e)$$



序列标注(Sequence Tagging)

以命名实体识别任务(NER)为例，对给定输入文本的每个词输出一个标签，以此指定某个命名实体的边界信息

原始标签	B-PER	I-PER	O	O	O	O	B-LOC	
原始输入	John	Smith	has	never	been	to	Harbin	
处理后的标签	B-PER	I-PER	O	O	O	O	B-LOC	B-LOC
处理后的输入	John	Smith	has	never	been	to	Ha	##rbin



Single Sentence

序列标注 (Sequence Tagging)

输入层

$$X = [\text{CLS}] x_1 x_2 \cdots x_n [\text{SEP}]$$

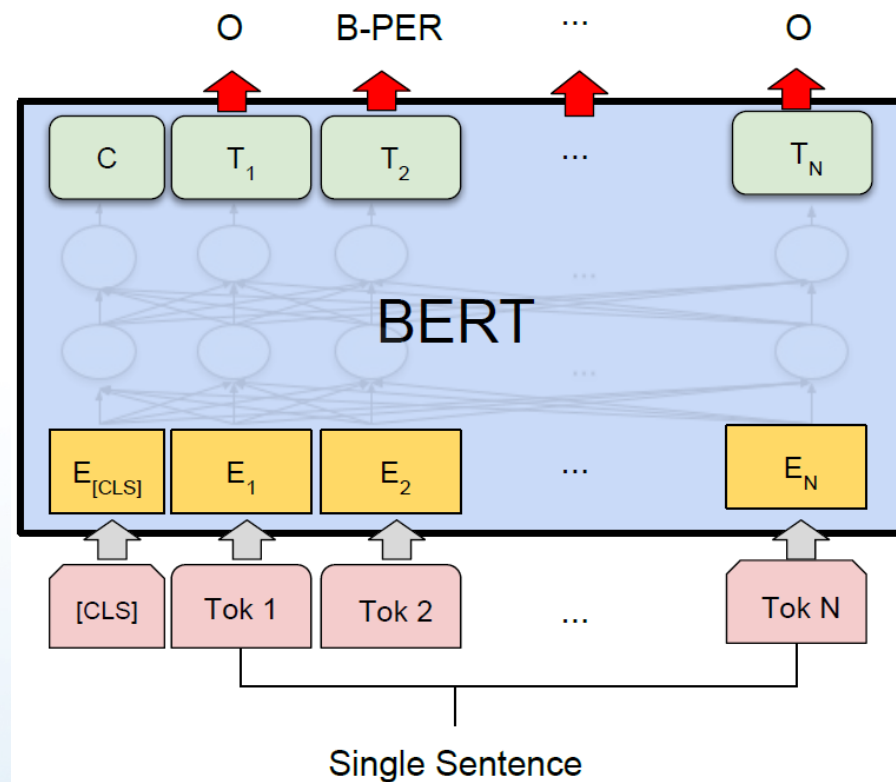
$$v = \text{InputRepresentation}(X)$$

BERT编码层

$$h = \text{BERT}(v)$$

序列标注层

$$P_t = \text{Softmax}(h_t W^o + b^o), \quad \forall t \in \{1, \cdots, N\}$$



得到每个位置对应的概率分布后，通过交叉熵求平均，得最终损失

BERT实验结果

- 该模型在机器阅读理解顶级水平测试SQuAD1.1中表现出惊人的成绩: 全部两个衡量指标上全面超越人类
- 并且在11种不同NLP测试中创出SOTA表现, 包括将GLUE基准推高至80.4% (绝对改进7.6%), MultiNLI准确度达到86.7% (绝对改进5.6%), 成为NLP发展史上的里程碑式的模型成就

目录

CONTENTS

1

概述

2

自回归预训练模型代表：GPT

3

自编码预训练模型代表：BERT

4

预训练语言模型的应用

5

深入理解BERT

深入理解BERT

- ❑ 以BERT、GPT等为代表的预训练技术为自然语言处理领域带来了巨大的变革
- ❑ 为了能够从大规模数据中充分地汲取知识，作为“容器”的预训练模型通常也需要具备相当大的规模。例如
 - ❑ BERT模型含有上亿个参数
 - ❑ OpenAI发布的GPT-3模型更是达到了惊人的千亿级参数
- ❑ 尽管这些大规模的预训练模型在很多任务上表现优异，但是庞大的模型体量也使得其预测行为变得更加难以“理解”以及“不可控”
- ❑ 很多实际应用中
 - ❑ 模型的性能重要
 - ❑ 模型行为给出可信的解释同样重要，大致衍生出两大类“可解释性”方面的研究
 - ❑ “自解释”的模型
 - ❑ “事后解释”

深入理解BERT

□可解释性

- 白解释：在模型构建之初针对性地设计其结构，使其具备可解释性

- 事后解释：对于 BERT 等大规模预训练模型的解释性研究，集中在此类

- “解释性”实际上是以人类的视角理解模型的行为。因此，需要建立模型的行为与人类概念系统之间的映射。

- 在NLP任务中，最具解释性的人类概念系统无疑是语言学特征。例如，

- BERT作为一个多任务通用的编码器，能够表达哪些语言学特征？

- BERT模型每一层使用的多头注意力又分别捕获了哪些关系特征？

- BERT模型的每一层表示是否和ELMo一样具有层次性？

□ 分析 BERT 模型的两个角度

- 自注意力机制：可视化分析的方式分析 BERT 的自注意力机制
- 表示学习：探针” 实验分析方法

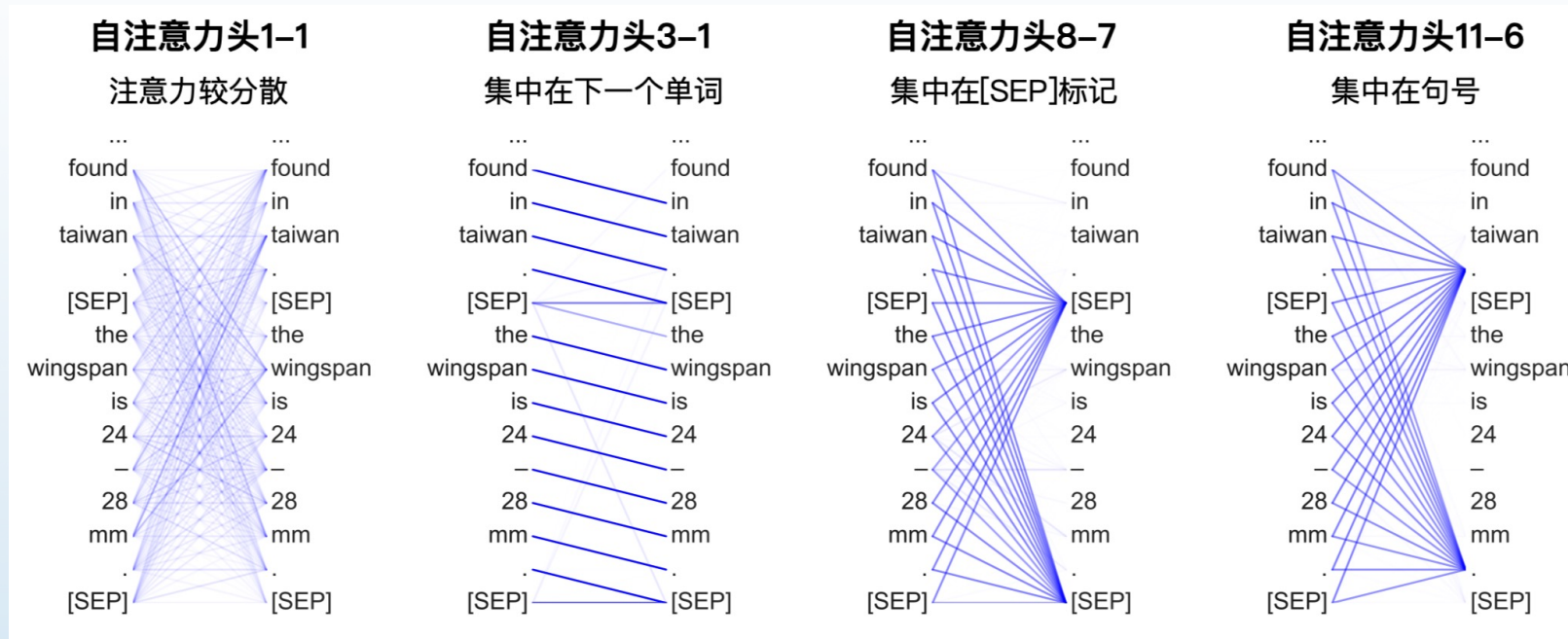
深入理解BERT——自注意力可视化分析

- ❑ BERT模型依赖Transformer 结构，其主要由多层自注意力网络层堆叠而成(含残差连接)
- ❑ 自注意力的本质事实上是对词(或标记)与词之间关系的刻画
- ❑ 不同类型的关系可以表达丰富的语义，例如
 - ❑ 名词短语内的依存关系
 - ❑ 句法依存关系
 - ❑ 指代关系
- ❑ 这些关系特征对于大部分语义理解类NLP任务具有关键的作用
- ❑ 因此，自注意力的分析将有助于理解BERT模型对于关系特征的学习能力

□ BERT不同自注意力头的行为模式对比

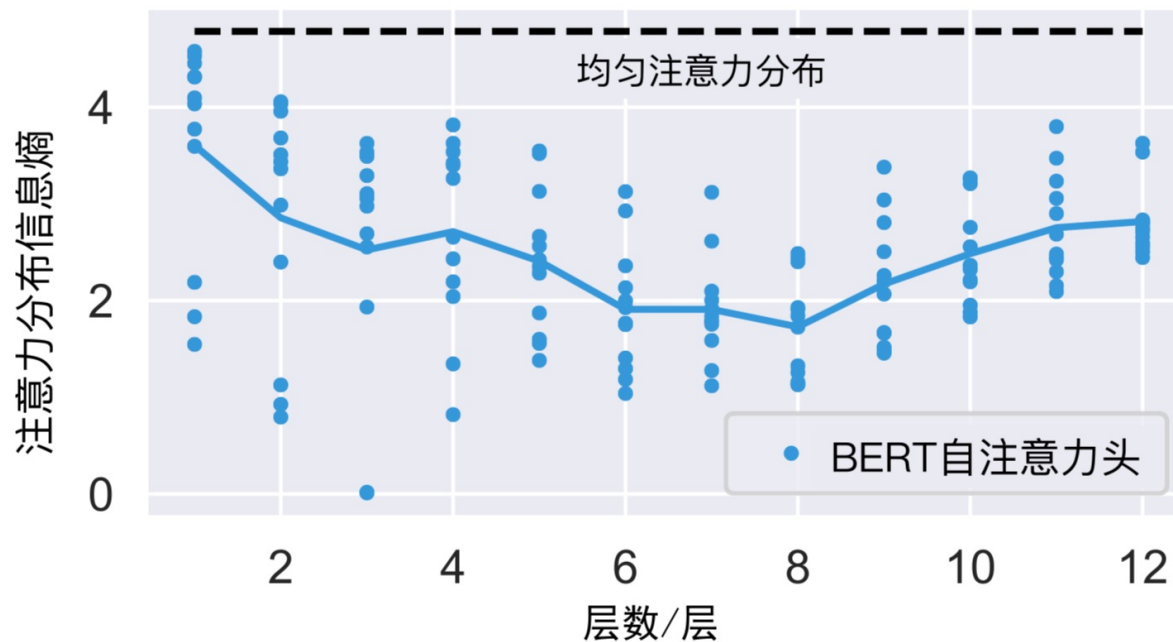
□ 文献[25]随机选取了1,000个维基百科文本片段，并对BERT多头自注意力进行了分析。下图可视化结果展示了不同自注意力头的行为：

- 有些注意力头分布较为均匀，具有较大的感受野，即编码了较“分散”的上下文信息；
- 有些注意力头的注意力分布较为集中，且显示出一定的模式，如集中在当前词的下一个词，或者[SEP]、句号等标记上。
- 不同的注意力头具有比较多样化的行为，因而能够编码不同类型的上下文和关系特征。



(x-y表示第x层的第y个自注意力头)

文献作者进一步分析了**不同层的注意力**分布：计算各层注意力分布的**信息熵**



熵的变化趋势

- 一部分注意力头分布具有较大的熵值(接近平均分布)，尤其在BERT的浅层
 - 较深的自注意力层(如6~8层)，其分布相对集中，熵值较小
 - 当接近输出层时，熵值又增大
 - 这种变化趋势在一定程度上可以反映BERT模型中信息聚合(或语义组合)的过程
- 在注意力分布较为“广泛”的模型浅层，其表示接近于**词袋表示**
- 随着层次变深，信息开始以不同的方式组合，从而形成**集中在不同局部**的注意力分布
- 接近输出层的自注意力分布**与目标预训练任务直接相关。对于BERT而言，即为掩码语言模型（MLM）与下一个句子预测（NSP）的联合训练任务。
- 对更多关于BERT自注意力模式分析感兴趣的读者，请参考原文献。

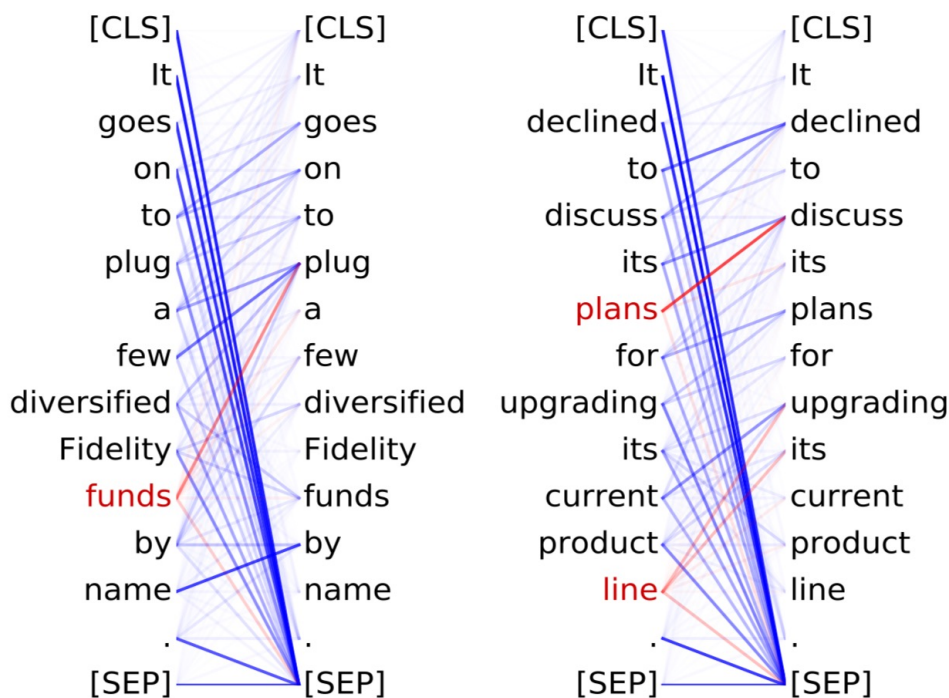
□ 探针实验

- 自注意力的可视化分析有助于从直观上理解模型内部的信息流动。而为了更准确地理解模型的行为，仍然需要定量的实验分析。
- 目前被广为采用的定量分析方法是探针实验
 - 探针实验的核心思想：设计特定的探针，对于待分析对象(如自注意力或隐含层表示)进行特定行为分析
 - 探针通常是一个非参或者非常轻量的参数模型(如线性分类器)，它接受待分析对象作为输入，并对特定行为预测
 - 预测的准确度可作为待分析对象是否具有该行为的衡量指标

深入理解BERT

□ 探针实验

- 为了检验某个自注意力头对直接宾语(Direct object, dobj)关系的表达能力，可以设计一个探针对该自注意力头在dobj句法关系预测上的表现进行分析
- 在BERT第8层第10个自注意力头(记为8-10号)的注意力分布中，其中红色高亮部分即为dobj关系



- 文献[25]在宾州依存树库(PTB)上进行了探针实验
 - 结果表明在BERT模型中，确实存在一部分自注意力头较好地捕捉到特定的句法关系
 - 例如，对于dobj关系的预测准确率达到86.8%。此外，对于更复杂的共指关系(Coreference)，同样能够找到具有较好预测能力的自注意力头。

深入理解BERT

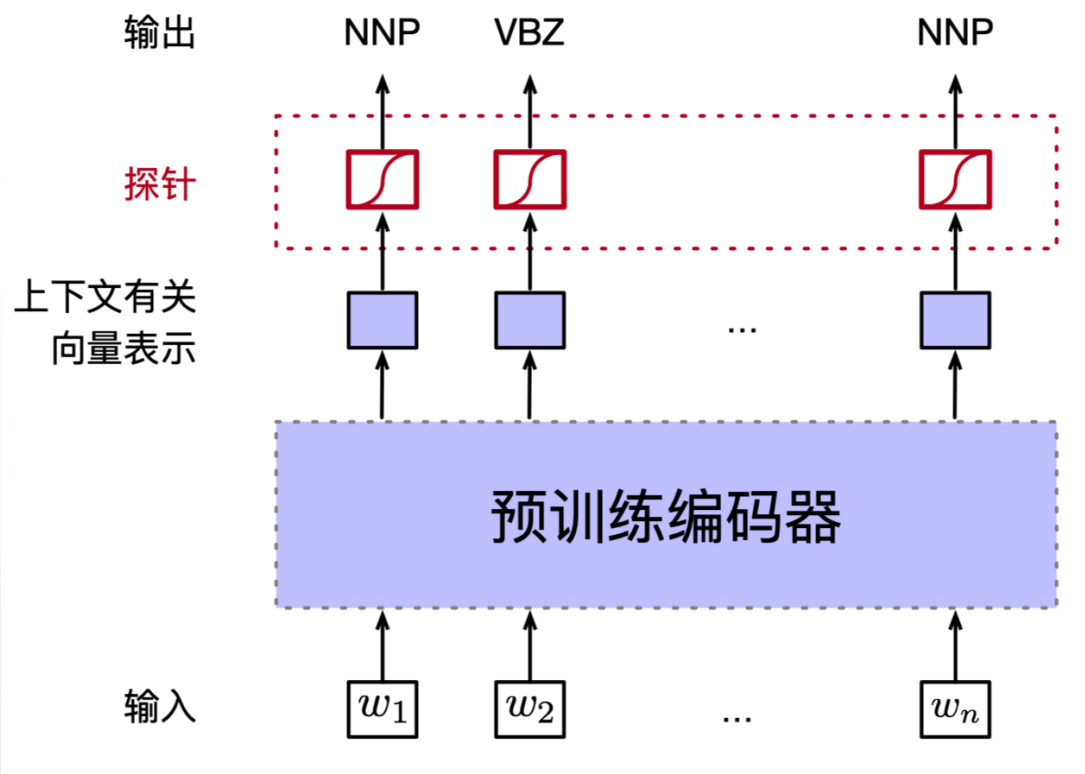
□ 探针实验

- 自注意力反映了预训练模型内部信息的聚合过程，而模型的各层隐含层表示是聚合的结果
- 对预训练编码器的隐含层表示直接进行探针实验，更好地理解其特性
- 探针可以是一个简单的线性分类器
- 该分类器利用模型的隐含层表示作为特征在目标任务（如词性标注）上训练，从而根据该任务的表现对预训练模型隐含层表示中蕴含的语言学特征评估

深入理解BERT

□ 探针实验

□ 探针的一般性框架



□ 对于更复杂的结构预测类任务，如句法分析等，也可以设计针对性的结构化探针。感兴趣的读者可以参考文献[26]。

Thanks!
Q&A ?

- wuxianyan@njau.edu.cn